



INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO
ESCOM

Trabajo Terminal

**Plataforma Web para la Denuncia Segura y Prevención de Riesgos
Digitales**

2026-B009

Presentan

Edgar Jair Flores Espinosa

Jose Bryan Omar Jimenez Velazquez

Leilani Shareni Santos Rivera

Directores

M. en C. Ulises Velez Saldaña

Dr. Jessie Paulina Guzmán Flores

Advertencia

“Este documento contiene información desarrollada por la Escuela Superior de Cómputo del Instituto Politécnico Nacional, a partir de datos y documentos con derecho de propiedad y por lo tanto, su uso quedará restringido a las aplicaciones que explícitamente se convengan.”

La aplicación no convenida exime a la escuela su responsabilidad técnica y da lugar a las consecuencias legales que para tal efecto se determinen.

Información adicional sobre este reporte técnico podrá obtenerse en:

La Subdirección Académica de la Escuela Superior de Cómputo del Instituto Politécnico Nacional, situada en Av. Juan de Dios Bátiz s/n Teléfono: 57296000, extensión 52000.

Agradecimientos

A mis padres, por su apoyo incondicional y por ser el pilar fundamental en mi formación académica y personal.

A mis directores, el M. en C. Ulises Vélez Saldaña y la Dra. Jessie Paulina Guzmán Flores, por su invaluable guía, conocimiento y por confiar en este trabajo terminal.

Al Instituto Politécnico Nacional y a la Escuela Superior de Cómputo, por brindarme las herramientas y el entorno necesario para formarme como profesional.

*PONER LO QUE USTEDES QUIERAN, LO DE ARRIBA LO COPIE
JAJAJAJA*

Índice General

Glosario	VII
1. Marco Teórico	1
1.1. Problema de clasificación en el contexto de la DDP	1
1.1.1. Taxonomía de denuncias	2
1.1.2. Clasificación binaria y lógica de admisión	3
1.1.3. Ciclo de vida de los datos en el aprendizaje supervisado	4
1.2. Fundamentos de Procesamiento de Lenguaje Natural	5
1.2.1. Álgebra lineal en espacios vectoriales de alta dimensión	5
1.2.2. Preprocesamiento avanzado	6
1.3. Representación Vectorial del Texto	7
1.3.1. Modelos de Bolsa de Palabras y TF-IDF	7
1.3.2. Word Embeddings Estáticos — spaCy (es_core_news_lg)	8
1.3.3. Sentence Transformers — SBERT (paraphrase-multilingual-mpnet-base-v2)	9
1.4. Ingeniería de Características Híbridas	10
1.4.1. Codificación de variables normativas	11
1.4.2. Concatenación y Factor de Amplificación (α)	12
1.5. Modelos de Aprendizaje Automático Evaluados	13
1.5.1. Máquinas de Soporte Vectorial (SVM)	13
1.5.2. Regresión Logística	14
1.5.3. Ensamble de Árboles — Random Forest	15
1.6. Estrategias de Entrenamiento y Validación Robusta	15
1.6.1. Validación Cruzada K -Fold Estratificada	16
1.6.2. Evaluación mediante Métricas de Inteligencia	16
1.6.3. Carga Perezosa y Serialización con joblib	17
1.7. Fundamentos de arquitectura de software	18
1.7.1. Definición de arquitectura de software	18

1.7.2.	Importancia en el Desarrollo de Sistemas	19
1.7.3.	Factores de calidad	19
1.8.	Arquitectura del Sistema	20
1.8.1.	Visión general de la arquitectura híbrida	20
1.8.2.	Arquitectura de Microservicios	20
1.8.3.	Arquitectura interna de los servicios: Patrón de 3 capas	22
1.8.4.	Servidor web y proxy inverso (Nginx)	23
1.9.	Tecnologías de Desarrollo Backend	24
1.9.1.	Lenguaje de programación: Java	24
1.9.2.	Framework: Spring Boot	25
1.9.3.	Interfaces de Programación (API REST): Estándares de interoperabilidad, documentación y versionamiento	30
1.10.	Seguridad y Comunicación de Datos (En Tránsito)	31
1.10.1.	Protocolos de Transporte Seguro: TLS 1.3	31
1.10.2.	Resiliencia del Sistema: Patrones API Gateway (Spring Cloud Gateway) y Circuit Breaker (Resilience4j)	32
1.11.	Infraestructura y Orquestación	34
1.11.1.	Contenerización: Fundamentos de Docker, Dockerfile y gestión con Docker Compose	34
1.11.2.	Orquestación de Contenedores: Kubernetes (K8s), gestión de Pods, Deploy- ments y escalabilidad horizontal (HPA)	35
1.12.	Gestión de Bases de Datos	35
1.12.1.	PostgreSQL	35
1.13.	Persistencia y Gestión del Repositorio Digital (En Reposo)	36
1.13.1.	PostgreSQL Avanzado: Manejo de datos semi-estructurados (JSONB) e indexación GIN para búsqueda de texto completo (Full-text search)	36
1.13.2.	Seguridad y Cifrado en Reposo: Implementación de pgcrypto (AES-256) y validación de integridad mediante hashes SHA-512	37
1.13.3.	Escalabilidad de Base de Datos: Particionamiento de tablas, pooling de conexiones (PgBouncer) y replicación streaming	38

Índice de figuras

1.1. Comunicación entre microservicios.	21
1.2. Patron de Diseño de 3 capas.	22
1.3. Visualización de los Endpoints de monitoreo proporcionados por Spring Boot Actuator.[?]	27
1.4. Uso de JwtAuthenticationProvider en el flujo de autenticación de Spring Security [?].	28

Índice de tablas

1.1. Features jurídicos estructurados derivados del Artículo 4º.	11
--	----

Glosario

Acuerdo de Conclusión: Documento final que cierra el expediente, pudiendo resultar en una recomendación, un cese de actos o un pronunciamiento de improcedencia.

Acuse de Recibo: Comprobante digital generado por el sistema que confirma que la DDP ha recibido la solicitud, otorgando un folio único de seguimiento.

Análisis de Patrones: Uso de algoritmos para identificar comportamientos repetitivos o anomalías (como focos de corrupción) dentro de grandes volúmenes de datos.

Análisis Bidimensional: Evaluación de datos cruzando dos variables distintas (ej. tipo de queja vs. ubicación geográfica) para obtener información estratégica.

Anonimato Bidireccional: Capacidad de un sistema para permitir la comunicación entre denunciante e investigador sin que ninguna de las partes conozca la identidad real de la otra.

API REST: Interfaz de programación de aplicaciones que utiliza peticiones HTTP para transferir datos entre componentes de software.

Arquitectura de Cero Conocimiento (Zero-Knowledge): Diseño de seguridad donde el servidor no tiene acceso a las llaves de cifrado del usuario, siendo incapaz de ver los datos almacenados.

Arquitectura de Microservicios: Estilo arquitectónico que estructura una aplicación como un conjunto de servicios pequeños, autónomos y acoplados de forma mínima.

Arquitectura Monolítica: Modelo de software donde todos los componentes del sistema están entrelazados en un solo programa, dificultando su escalabilidad.

AUC-ROC: Curva que mide la capacidad del modelo para distinguir entre clases; un área cercana a 1 indica un desempeño óptimo en la clasificación.

Autenticación Robusta: Proceso de validación de identidad que utiliza credenciales institucionales verificables (Boleta o Número de Empleado).

Autonomía Técnica: Capacidad de la DDP para emitir criterios y recomendaciones de forma independiente, sin influencia de otras autoridades.

Big Data: Conjuntos de datos tan grandes y complejos que requieren aplicaciones informáticas no tradicionales para su procesamiento.

Bolsa de Palabras (BoW): Técnica de representación de texto basada en la frecuencia de aparición de cada palabra, sin considerar el orden gramatical.

Caducidad / Rezago Administrativo: Situación que ocurre cuando un trámite no se realiza dentro de los plazos legales, afectando la eficiencia institucional.

Catálogo de Derechos Vulnerados: Listado estructurado de los derechos establecidos en la Ley Orgánica que sirve de base para la clasificación mediante PLN.

Cifrado de Extremo a Extremo (E2EE): Sistema de comunicación donde solo los usuarios finales pueden leer los mensajes, protegiendo la información de intermediarios.

Clasificador Neuro-simbólico: Enfoque de IA que combina redes neuronales (para aprendizaje) con lógica basada en reglas (para razonamiento).

Comunidad Politécnica: Conjunto de estudiantes, docentes, personal administrativo y egresados del IPN.

Competencia: Facultad o atribución legal que tiene un órgano (DDP) para conocer y resolver un asunto específico basado en su normativa.

Crowdsourcing: Modelo de obtención de servicios o ideas mediante la colaboración abierta de una comunidad o grupo de personas.

DDP (Defensoría de los Derechos Politécnicos): Órgano encargado de velar por los derechos de la comunidad del IPN.

Debido Proceso: Conjunto de formalidades esenciales que deben observarse en cualquier procedimiento legal para asegurar un resultado justo.

Disponibilidad: Capacidad de un sistema de permanecer operativo y accesible durante un periodo de tiempo determinado.

Escalabilidad: Propiedad de un sistema para manejar una cantidad creciente de trabajo o su potencial para ser ampliado.

Estado del Arte: Análisis de los conocimientos, tecnologías y soluciones más avanzados que existen actualmente sobre un tema de investigación.

Expediente Digital: Repositorio electrónico que centraliza documentación, evidencias multimedia y actuaciones jurídicas de un caso.

F1-Score: Métrica reina que representa el promedio balanceado entre Precisión y Recall, útil para evaluar la calidad global del modelo.

Flujo Operativo: Secuencia estandarizada de etapas por las que debe pasar una queja, desde su recepción hasta su archivo definitivo.

Georreferenciación / Geolocalización: Técnica que permite asignar una ubicación geográfica exacta (coordenadas) a un reporte o evento.

Grid Search: Búsqueda exhaustiva para encontrar los mejores hiperparámetros de un modelo probando todas las combinaciones posibles.

GRC (Governance, Risk, and Compliance): Estrategia para gestionar el gobierno corporativo, los riesgos y el cumplimiento legal de una organización.

Hiperplano: Frontera o muro matemático en un espacio multidimensional que separa las distintas clases (ej. quejas competentes de improcedentes).

Independencia Condicional: Supuesto del algoritmo Naive Bayes donde se asume que la presencia de una palabra no afecta a otra, simplificando el cálculo de probabilidades.

Infraestructura On-Premise: Modelo de despliegue donde los servidores y el software se encuentran físicamente dentro de las instalaciones de la institución.

Inferencia Bayesiana: Modelo estadístico de probabilidad que actualiza la estimación de un fenómeno a medida que se adquiere más evidencia.

Inmutabilidad de los Datos: Propiedad que asegura que la información registrada no puede ser alterada o borrada, garantizando la integridad del expediente.

Integridad Documental: Garantía de que los documentos que integran el expediente digital no han sido manipulados desde su carga original.

IPN (Instituto Politécnico Nacional): Institución pública mexicana de educación media superior, superior y posgrado.

Java Spring Boot: Framework diseñado para simplificar el desarrollo de aplicaciones Java basadas en microservicios.

Justicia Restaurativa: Enfoque de justicia centrado en reparar el daño causado a las víctimas, en lugar de solo castigar al infractor.

Ley Orgánica: Marco legal máximo que rige la estructura, fines y organización del Instituto Politécnico Nacional.

Manual de Organización / Procedimientos: Documentos normativos que definen las funciones y los pasos operativos internos de la Defensoría.

Margen: Distancia entre los vectores de soporte y el hiperplano; el sistema busca maximizar esta distancia para reducir errores de clasificación.

Matriz de Confusión: Tabla que permite visualizar el desempeño de un algoritmo, mostrando los aciertos y las confusiones (falsos positivos y negativos).

Máxima Verosimilitud (MLE): Método para estimar las probabilidades de un modelo basándose en la frecuencia de aparición de los datos en el entrenamiento.

Medidas de Protección: Acciones urgentes dictadas para salvaguardar la integridad de la persona quejosa durante la investigación.

Naive Bayes: Algoritmo probabilístico que clasifica quejas basándose en el contenido de sus palabras bajo un supuesto de independencia.

Oficio de Solicitud de Informe: Documento oficial para requerir a una autoridad que rinda su declaración sobre los hechos de una queja.

PLN / NLP (Procesamiento de Lenguaje Natural): Rama de la IA que permite a las máquinas entender, interpretar y manipular el lenguaje humano.

PostgreSQL: Sistema de gestión de bases de datos relacional orientado a objetos y de código abierto.

Precisión: Métrica que indica qué tan exacto es el sistema cuando clasifica una queja como positiva, evitando falsas alarmas.

Primer Contacto: Etapa inicial donde se recibe la solicitud y se realiza el filtrado preliminar de la queja.

Procedencia: Calificación de una solicitud que cumple con los requisitos legales para ser admitida y analizada de fondo.

Punto de Falla Único: Componente de un sistema que, en caso de fallar, provoca que todo el sistema deje de funcionar.

Quejoso: Persona que interpone una queja o denuncia por una presunta vulneración de sus derechos.

Radicación: Acto formal de registrar y asignar un número de expediente a una queja para iniciar su proceso legal.

RBAC (Role-Based Access Control): Método de control de acceso que restringe o permite funciones del sistema según el rol asignado al usuario.

Recall (Sensibilidad): Capacidad del sistema para detectar todos los casos positivos reales, asegurando que ninguna queja válida sea ignorada.

Reglas de Negocio: Políticas institucionales que el software debe ejecutar automáticamente para garantizar la validez del proceso.

SaaS (Software as a Service): Modelo de distribución de software donde los datos se alojan en servidores externos y se accede vía web.

Similitud de Coseno: Medida métrica utilizada para determinar qué tan parecidos son dos documentos de texto en un espacio vectorial.

SMOTE (Synthetic Minority Over-sampling Technique): Técnica de balanceo de datos que crea ejemplos sintéticos para clases minoritarias, permitiendo un aprendizaje parejo.

Soberanía de Datos: Principio que establece que los datos están sujetos a las leyes y control de la institución donde se generan.

Stemming y Lematización: Procesos lingüísticos para reducir palabras a su raíz o forma base (lema) para facilitar el análisis.

Suavizado de Laplace (α): Ajuste matemático que suma un valor pequeño a los conteos para evitar probabilidades de cero en palabras no vistas previamente.

SVM (Support Vector Machine): Algoritmo de aprendizaje supervisado que busca el hiperplano óptimo para clasificar datos en diferentes categorías.

Términos Procesales / Plazos Legales: Lapsos de tiempo obligatorios otorgados por la normativa para realizar una acción jurídica.

- TF-IDF:** Técnica de pesaje que resalta palabras clave de un problema y resta importancia a palabras comunes o irrelevantes.
- TLS (Transport Layer Security):** Protocolo criptográfico que proporciona comunicaciones seguras por red, cifrando los datos en tránsito.
- Tokenización:** Proceso de fragmentar una cadena de texto en unidades más pequeñas llamadas tokens (palabras o símbolos).
- Trazabilidad:** Capacidad de seguir el historial, la aplicación y la ubicación de un expediente a través de todas sus etapas.
- Triaje Administrativo:** Proceso de selección y priorización de quejas basado en urgencia, competencia legal y gravedad.
- Truco del Kernel (Kernel Trick):** Técnica que proyecta datos a dimensiones superiores para encontrar una separación lineal donde antes no era posible.
- Validación Cruzada (K-Fold):** Método de evaluación que divide los datos en K partes para asegurar que el sistema sea robusto con diferentes subconjuntos de datos.
- Variables de Holgura (ξ):** Parámetro de flexibilidad en un modelo SVM que permite que algunos puntos crucen la frontera para manejar datos con ruido.
- Vectores de Soporte:** Puntos o quejas que se encuentran más cerca de la frontera de decisión y definen la posición del hiperplano.
- Ventana de Contexto:** Rango de palabras que el modelo analiza alrededor de un término objetivo para aprender su significado semántico.
- Whistleblowing:** Acto de denunciar internamente o ante las autoridades actividades ilegales o poco éticas dentro de una organización.
- Word Embeddings:** Representación de palabras como vectores en un espacio multidimensional donde la cercanía geométrica indica cercanía semántica.
- Word2Vec / GloVe:** Modelos específicos utilizados para generar embeddings basados en el contexto local o estadísticas globales del texto.

Capítulo 1

Marco Teórico

Con el fin de optimizar el flujo operativo dentro de la Defensoría de Derechos Politécnicos (DDP), se implementará un sistema de clasificación para determinar la competencia o incompetencia de una queja. Este sistema brindará apoyo directamente en la etapa de primer contacto; con ello se busca transformar el análisis normativo en un modelo computacional capaz de identificar la competencia de manera consistente y eficiente.

1.1 Problema de clasificación en el contexto de la DDP

Este problema parte del marco legal presentado en el Acuerdo publicado en la Gaceta Politécnica, Número Extraordinario 622, donde se presenta a la DDP como el órgano encargado de promover, proteger y defender los derechos de la comunidad frente a actos u omisiones de las autoridades del Instituto.

En el Artículo 3° del mismo Acuerdo se establece que la Defensoría es competente para conocer quejas relacionadas con presuntas violaciones a los derechos de la comunidad politécnica; sin embargo, el Artículo 4° delimita los márgenes en las materias en que la Defensoría **no** es competente:

1. Afectaciones de carácter colectivo.
2. Resoluciones disciplinarias.
3. Derechos de naturaleza laboral.

4. Evaluaciones académicas de profesores, comisiones dictaminadoras o consejos técnicos consultivos escolares, salvo que en dicha evaluación se violen notoriamente los derechos de los interesados.
5. Procedimientos y resoluciones instaurados por el Órgano Interno de Control en el Instituto.

El proceso de recepción de quejas descrito en el Artículo 13 establece que el personal de la Defensoría debe determinar en primera instancia si la queja es competente o incompetente. Esta decisión, aparentemente sencilla, presenta en la práctica alta ambigüedad derivada de la redacción de los hechos. Dicha dependencia del criterio manual genera tres conflictos críticos:

1. El consumo excesivo de tiempo administrativo.
2. La variabilidad de interpretaciones ante casos similares.
3. La saturación operativa debida al incremento constante de solicitudes.

Con esto, el problema se define como una tarea de *Procesamiento de Lenguaje Natural* (PLN) y *clasificación supervisada*, donde el sistema debe aprender a identificar los patrones lingüísticos y normativos que separan una queja competente de una incompetente.

1.1.1 Taxonomía de denuncias

Lo anterior pareciera una tarea sencilla; no obstante, transformar una narrativa jurídica en un conjunto de datos procesable por algoritmos de aprendizaje automático requiere establecer una taxonomía clara que estructure el conocimiento normativo.

En el contexto de la Defensoría, las denuncias no se clasifican únicamente por su contenido textual, sino por una estructura jerárquica de validación que el sistema debe reconocer. Esta taxonomía se apoya en dos reglas críticas:

1. **Pertenencia a la comunidad y jerarquía.** Define si las personas involucradas pertenecen a la comunidad politécnica. El sistema deberá identificar la existencia de un quejoso con su estatus institucional y la autoridad a quien se imputa la violación. Sin esta validación de identidad, la queja carece de base para ser procesada.
2. **Exclusiones del Artículo 4°.** Constituye el núcleo del clasificador. Las denuncias se categorizan según la presencia de los cinco criterios de exclusión mencionados anteriormente. Esta regla se traduce en el sistema como siete campos binarios —características estructuradas— que procesan el análisis normativo:

- Naturaleza laboral vs. derechos politécnicos.
- Evaluación académica pura vs. vulneración de derechos en evaluación.
- Afectación individual vs. colectiva.
- Resolución disciplinaria vs. acto de autoridad.
- Procedimientos del OIC vs. acto u omisión de autoridad institucional.

Esta estructuración permite que el problema de clasificación no dependa únicamente del texto libre, sino de reglas establecidas en el Artículo 4°. Al final del proceso taxonómico, el sistema asigna a cada expediente una etiqueta binaria definitiva: *Competente* o *No Competente* (canalización a otra instancia o desechamiento), lo que constituye la variable de salida de los modelos de clasificación supervisada.

1.1.2 Clasificación binaria y lógica de admisión

Este clasificador se implementa bajo un esquema de **clasificación binaria**, decisión fundamentada directamente en el Artículo 13 del Acuerdo número extraordinario 622. A diferencia de otros sistemas de categorización temática —que discriminan entre dominios como deportes, política o finanzas— el clasificador de la DDP produce un veredicto dicotómico: *Competente* o *No Competente*.

Esta simplificación a dos clases responde a la lógica de admisión procesal por tres razones fundamentales:

1. **Naturaleza del flujo administrativo.** Dentro de la DDP, el primer contacto actúa como una compuerta. No existe un estado intermedio: una queja o es admitida para su investigación, o es desechada y canalizada a otra instancia. La clasificación binaria representa fielmente este proceso de toma de decisiones institucional.
2. **Optimización del entrenamiento.** Desde la perspectiva del aprendizaje automático, concentrar la muestra en dos etiquetas permite una mayor densidad de datos por categoría. Dado que el volumen de expedientes históricos etiquetados es finito, un modelo binario es más robusto y menos propenso al error que un modelo multiclase que intentara identificar simultáneamente el tipo específico de exclusión: laboral, académica o disciplinaria.
3. **Reducción del falso negativo.** La lógica de admisión prioriza la protección de los derechos de la comunidad. Al configurar el problema como binario, es posible ajustar el umbral de decisión del modelo —especialmente en algoritmos como SVM— para minimizar los falsos

negativos, asegurando que ninguna queja potencialmente competente sea descartada por una clasificación errónea.

Por lo tanto, si el modelo predice la clase *No Competente*, el sistema asiste al personal sugiriendo la fundamentación legal basada en el Artículo 4°. Si la predicción es *Competente*, el expediente fluye automáticamente hacia la etapa de radicación en la Subdefensoría, logrando así la meta de reducir los tiempos de respuesta institucional.

1.1.3 Ciclo de vida de los datos en el aprendizaje supervisado

La implementación de un clasificador inteligente requiere establecer un flujo técnico robusto que garantice que la narrativa jurídica se transforme en una señal numérica apta para el aprendizaje automático. En el contexto de este proyecto, el ciclo de vida de los datos se estructura en seis etapas fundamentales:

1. **Captura de datos.** El ciclo inicia en la interfaz de registro donde el quejoso proporciona de forma estructurada su identidad institucional y, de forma no estructurada, el relato de los hechos. Esta etapa asegura que la entrada cumpla con los requisitos mínimos de admisión definidos en el Artículo 13.
2. **Preprocesamiento y limpieza.** La narrativa libre se somete a técnicas de PLN. Se eliminan elementos de ruido, se normaliza el texto y se identifican términos clave que corresponden a las exclusiones normativas del Artículo 4°.
3. **Vectorización y enriquecimiento.** El texto se convierte en representaciones vectoriales (*embeddings*). Simultáneamente, el sistema inyecta las características jurídicas binarias extraídas del expediente, aplicando el factor de amplificación α para dar peso a la señal normativa sobre la semántica.
4. **Entrenamiento y ajuste.** Utilizando un conjunto de datos histórico previamente etiquetado, el modelo aprende a identificar las fronteras de decisión legal. Se emplean técnicas de validación cruzada para medir la robustez de la generalización.
5. **Inferencia y sugerencia.** Ante una queja nueva, el modelo procesa los vectores y genera una predicción de competencia acompañada de un nivel de probabilidad. El sistema asiste al personal humano mostrando el veredicto sugerido y la posible fundamentación legal.
6. **Retroalimentación.** El ciclo se cierra cuando el personal jurídico valida o corrige la sugerencia del modelo. Estas decisiones se reincorporan al repositorio de datos para el reentrenamiento

futuro, permitiendo que el clasificador evolucione conforme cambian los criterios institucionales.

Este ciclo asegura que el sistema no opere como una herramienta estática, sino que mantenga la trazabilidad y la integridad de la información jurídica en cada una de sus fases.

1.2 Fundamentos de Procesamiento de Lenguaje Natural

El Procesamiento de Lenguaje Natural (PLN) es la rama de la inteligencia artificial que dota a las computadoras de la capacidad de comprender, interpretar y generar lenguaje humano. En el contexto jurídico, el PLN enfrenta desafíos particulares: el vocabulario especializado, la estructura argumentativa implícita y la alta dependencia del contexto normativo hacen que dos quejas léxicamente similares puedan tener clasificaciones opuestas.

1.2.1 Álgebra lineal en espacios vectoriales de alta dimensión

El fundamento matemático sobre el que descansan todas las representaciones textuales empleadas en este sistema es el **espacio vectorial de alta dimensión**. La idea central es que cualquier documento puede representarse como un punto en un espacio $\mathbb{R}^d$, donde d es el número de dimensiones del vector. Documentos semánticamente similares ocupan regiones cercanas en dicho espacio.

Sea $\mathbf{v} \in \mathbb{R}^d$ un vector de representación de un documento. La **similitud coseno** entre dos documentos $\mathbf{u}$ y $\mathbf{v}$ se define como:

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\| \|\mathbf{v}\|} = \frac{\sum_{i=1}^d u_i v_i}{\sqrt{\sum_{i=1}^d u_i^2} \cdot \sqrt{\sum_{i=1}^d v_i^2}} \quad (1.1)$$

donde $\text{sim} \in [-1, 1]$. Un valor cercano a 1 indica alta similitud semántica.

En este proyecto se trabaja con espacios de $d = 300$ dimensiones (spaCy) y $d = 768$ dimensiones (Sentence-BERT). La diferencia en dimensionalidad no es trivial: a mayor dimensión, el modelo puede capturar matices semánticos más finos, pero también requiere mayor capacidad computacional y datos suficientes para evitar la *maldición de la dimensionalidad* (el volumen del

espacio aumenta exponencialmente con cada nueva dimensión, lo que provoca que los datos se vuelvan muy dispersos).

Las operaciones fundamentales que el sistema ejecuta sobre estos espacios son:

- **Suma de vectores:** permite combinar representaciones de tokens individuales en una representación de documento.
- **Promedio ponderado:** base del vectorizador de spaCy.
- **Producto punto y norma:** usados internamente por SVM para calcular márgenes de separación.
- **Concatenación horizontal:** operación central de la arquitectura híbrida para unir embeddings de texto con features jurídicos.

1.2.2 Preprocesamiento

avanzado

Antes de convertir el texto en vectores, la narrativa libre del quejoso debe pasar por una cadena de transformaciones que reduzcan el ruido y estandaricen la representación lingüística. Este proceso es especialmente relevante en textos jurídicos, donde variaciones ortográficas, errores tipográficos o sinónimos coloquiales de términos normativos pueden afectar la calidad de los embeddings.

Tokenización

La **tokenización** segmenta el texto continuo en unidades mínimas de significado (tokens). spaCy realiza una tokenización sensible al contexto que distingue, por ejemplo, contracciones y signos de puntuación:

"me negó el acceso" → [me, negó, el, acceso]

Lematización

La **lematización** reduce cada token a su forma canónica (*lema*), eliminando variaciones morfológicas:

negaron → negar, discriminado → discriminar, estudiantiles → estudiantil

Filtrado de palabras vacías

Las **palabras vacías** (*stop words*) son términos de alta frecuencia y bajo contenido semántico (artículos, preposiciones, conjunciones). En textos jurídicos, sin embargo, ciertas palabras que en otros contextos serían vacías — como “no”, “sin” o “nunca” — pueden ser jurídicamente decisivas. spaCy permite personalizar esta lista para el dominio específico de la Defensoría.

Análisis morfosintáctico con spaCy

El modelo `es_core_news_lg` de spaCy realiza, además de la tokenización y lematización, un análisis morfosintáctico completo que asigna a cada token su categoría gramatical (*POS tagging*) y sus dependencias sintácticas. Esto permite identificar, por ejemplo, que en la oración:

“El director me negó el acceso al laboratorio”

el sujeto es *director* (autoridad), el verbo es *negar* (acto de omisión) y el complemento es *acceso al laboratorio* (derecho afectado). Esta información estructural enriquece la representación semántica más allá del léxico superficial.

1.3 Representación Vectorial del Texto

La representación vectorial del texto es el puente entre el lenguaje natural y los algoritmos de aprendizaje automático. A continuación se presenta la evolución de estas técnicas desde los modelos estadísticos hasta los modelos de lenguaje contextuales de última generación.

1.3.1 Modelos de Bolsa de Palabras y TF-IDF

La caracterización de bolsa de palabras puede ser considerado una forma de procesamiento de texto sencilla por su aparente simplicidad en recuento de palabras en un conjunto de textos. En bolsa de palabras, cada palabra individual se convierte en una dimensión separada del espacio vectorial. Si un conjunto de texto tiene n cantidad de palabras, el espacio vectorial resultante tendrá n dimensiones. El modelo trazará cada documento de texto por separado como un punto en el espacio vectorial. La posición de un punto está determinada por la cantidad de veces que aparece la palabra de esa dimensión dentro del documento del punto.

Los modelos de **Bolsa de Palabras** (*Bag of Words*, BoW) son el punto de partida histórico de la representación vectorial. Representan un documento como un vector de frecuencias de términos sobre el vocabulario total del corpus:

$$\mathbf{v}_{BoW}(d) = [f(t_1, d), f(t_2, d), \dots, f(t_V, d)] \quad (1.2)$$

donde $f(t_i, d)$ es la frecuencia del término t_i en el documento d y V es el tamaño del vocabulario.

TF-IDF (*Term Frequency – Inverse Document Frequency*) refina este enfoque ponderando cada término según su frecuencia en el documento y su rareza en el corpus:

$$\text{tf-idf}(t, d, D) = \underbrace{\frac{f(t, d)}{\sum_{t'} f(t', d)}}_{\text{TF}} \times \underbrace{\log \frac{|D|}{|\{d' \in D : t \in d'\}|}}_{\text{IDF}} \quad (1.3)$$

Limitaciones críticas para el dominio jurídico. Aunque son computacionalmente eficientes, estos modelos son inadecuados para clasificar quejas de la DDP por dos razones fundamentales:

1. **Pérdida del orden y contexto:** Las frases “*el director negó el acceso*” y “*el acceso negó al director*” generan el mismo vector, aunque tienen significados radicalmente distintos.
2. **Sesgo léxico superficial:** El término “*calificación*” tendría alta frecuencia tanto en quejas por evaluación académica (No Competente) como en quejas por extorsión académica (Competente), haciendo imposible la distinción correcta.

Estos modelos se presentan como antecedentes necesarios para contextualizar la superioridad de los enfoques basados en embeddings empleados en este sistema.

1.3.2 Word Embeddings Estáticos — spaCy (es_core_news_lg)

Los **embeddings estáticos** resuelven la limitación semántica de BoW al representar cada palabra como un vector denso en $\mathbb{R}^{300}$, entrenado sobre grandes corpus en español mediante el algoritmo *word2vec*.^[?] Palabras semánticamente relacionadas —como *acoso* y *hostigamiento*— ocupan regiones cercanas en este espacio.

El modelo `es_core_news_lg` de spaCy [?] fue entrenado sobre el corpus *Spanish Web Crawl* y genera vectores de 300 dimensiones por token. La representación de un documento completo se obtiene promediando los vectores de los tokens informativos:

$$\mathbf{v}_{spaCy}(d) = \frac{1}{|T|} \sum_{t \in T} \mathbf{w}_t, \quad T = \{t \in d \mid \text{has_vector}(t) \wedge \neg \text{is_stop}(t)\} \quad (1.4)$$

donde $\mathbf{w}_t \in \mathbb{R}^{300}$ es el vector del token t y T es el conjunto de tokens informativos (con vector disponible y no vacías).

Al promediar todos los vectores del documento se pierde el orden sintáctico y el contexto local. La frase “*el profesor me acosó condicionando la calificación*” y la frase “*el profesor condicionó la calificación — sin acosar*” generarían vectores cercanos, a pesar de tener clasificaciones jurídicas distintas.

1.3.3 Sentence Transformers — SBERT (paraphrase-multilingual-mpnet-base-v2)

Sentence-BERT (SBERT) representa el estado del arte en codificación semántica de textos. [?] A diferencia de los embeddings estáticos, SBERT genera un único vector de representación para toda la oración, capturando las relaciones entre palabras a lo largo del texto completo mediante la arquitectura *Transformer*.

Arquitectura

Transformer

El bloque central del Transformer es el mecanismo de **atención multi-cabeza** (*multi-head self-attention*), que calcula para cada token su relevancia en función de todos los demás tokens del documento:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) V \quad (1.5)$$

donde Q , K y V son las matrices de consulta, clave y valor derivadas de los embeddings de entrada, y d_k es la dimensión de la clave. [?] Este mecanismo permite que el modelo asigne mayor peso a los tokens contextualmente relevantes. Por ejemplo, en la queja:

“Me reprobó porque no acepté salir con él”

el mecanismo de atención aprende a conectar *reprobó* con *acepté salir*, identificando que la relación causal entre ambos constituye un acto de acoso, no una evaluación académica pura.

Entrenamiento contrastivo de SBERT

SBERT extiende BERT con una capa de *pooling* sobre los estados ocultos y se entrena mediante aprendizaje contrastivo con pares de oraciones semánticamente equivalentes y no equivalentes:

$$\mathcal{L}_{contrastiva} = \sum_{(i,j)} [\text{sim}(\mathbf{e}_i, \mathbf{e}_j) - y_{ij}]^2 \quad (1.6)$$

donde $y_{ij} = 1$ si las oraciones i y j son semánticamente equivalentes y $y_{ij} = 0$ en caso contrario.

Modelo empleado

El modelo *paraphrase-multilingual-mpnet-base-v2* fue entrenado sobre más de 50 idiomas, incluido el español. Genera embeddings de **768 dimensiones** por documento y ha demostrado superioridad frente a los promedios de *word embeddings* en tareas de clasificación de texto corto y mediano. Esta capacidad es crítica para distinguir los casos límite de la DDP, donde el acto jurídicamente relevante puede estar expresado en una sola oración dentro de un párrafo más extenso.

1.4 Ingeniería de Características Híbridas

Una de las contribuciones centrales de este sistema es la **arquitectura híbrida** que combina la representación semántica del texto con conocimiento normativo estructurado. Esto responde a una limitación fundamental de los modelos puramente textuales: pueden aprender correlaciones léxicas espurias en lugar de la lógica jurídica subyacente.

1.4.1 Codificación de variables normativas

Los cinco criterios de incompetencia del Artículo 4° se traducen en **siete variables binarias** (features jurídicos), que codifican el análisis normativo de cada queja de forma estructurada:

Tabla 1.1: Features jurídicos estructurados derivados del Artículo 4°.

# Variable	Descripción normativa
1 es_conflicto_laboral_art4_I	El fondo del conflicto es una relación laboral o sindical (salarios, plazas, escalafón, prestaciones).
2 es_evaluacion_academica_pura_art4_II	La queja versa <i>únicamente</i> sobre la valoración del aprendizaje por parte de un profesor o comisión.
3 hay_vulneracion_notoria_en_evaluacion	Existe una violación de derechos dentro de un proceso evaluativo (extorsión, acoso, discriminación).
4 es_resolucion_disciplinaria_art4_III	La queja impugna una resolución disciplinaria ya emitida.
5 es_afectacion_colectiva_art4_IV	La afectación descrita es grupal, no individual.
6 involucra_oic_art4_V	El asunto está siendo conocido por el Órgano Interno de Control.
7 hay_acto_u_omision_autoridad	Existe un acto u omisión de una autoridad institucional que vulnera derechos políticos.

Estas variables se representan como un vector $\mathbf{f} \in \{0, 1\}^7$ y se determinan a partir del análisis del expediente, ya sea de forma manual durante la construcción del corpus o de forma automática en una etapa futura mediante un modelo extractor.

La lógica de consistencia entre las variables y la clase asignada es la siguiente:

- Si clase = No Competente por razón laboral: es_conflicto_laboral_art4_I = 1, resto = 0.

- Si `clase = Competente`: `hay_acto_u_omision_autoridad = 1`, todas las fracciones del Art. 4° = 0.
- La variable 3 (`hay_vulneracion_notoria`) solo puede ser 1 cuando la variable 2 (`es_evaluacion_academica_pura`) también es 1 y la clase es *Competente*: esto modela la excepción de la fracción IV del Artículo 4°.

1.4.2 Concatenación y Factor de Amplificación (α)

La unión del embedding semántico $\mathbf{e} \in \mathbb{R}^d$ con el vector jurídico $\mathbf{f} \in \{0, 1\}^7$ se realiza mediante **concatenación horizontal**:

$$\mathbf{x}_{final} = [\mathbf{e} \parallel \alpha \cdot \mathbf{f}] \in \mathbb{R}^{d+7} \quad (1.7)$$

donde $d \in \{300, 768\}$ según el vectorizador empleado y α es el **factor de amplificación**.

Justificación matemática del factor α

Sin amplificación ($\alpha = 1$), los 7 features jurídicos quedan efectivamente opacados por las d dimensiones del embedding, cuya norma promedio es significativamente mayor. Para ilustrarlo, considérese la contribución relativa de cada componente a la norma del vector concatenado:

$$\frac{\|\alpha \cdot \mathbf{f}\|}{\|\mathbf{e}\| + \|\alpha \cdot \mathbf{f}\|} = \frac{\alpha\sqrt{7}}{\|\mathbf{e}\| + \alpha\sqrt{7}} \quad (1.8)$$

Con $\|\mathbf{e}\| \approx 15$ (valor típico para embeddings de Sentence-BERT normalizados) y $\alpha = 1$:

$$\frac{1 \cdot \sqrt{7}}{15 + \sqrt{7}} \approx \frac{2,65}{17,65} \approx 15 \%$$

Con $\alpha = 3$:

$$\frac{3\sqrt{7}}{15 + 3\sqrt{7}} \approx \frac{7,94}{22,94} \approx 35 \%$$

El valor $\alpha = 3,0$ fue seleccionado experimentalmente como el punto de equilibrio donde la señal jurídica tiene peso suficiente para influir en las fronteras de decisión del clasificador, sin dominar completamente sobre la información semántica del texto.

1.5 Modelos de Aprendizaje Automático Evaluados

Para garantizar la selección del clasificador óptimo para el dominio jurídico de la DDP, se evaluaron tres familias de algoritmos de aprendizaje supervisado con distintos paradigmas de construcción de fronteras de decisión.

1.5.1 Máquinas de Soporte Vectorial (SVM)

Las **Máquinas de Soporte Vectorial** (*Support Vector Machines*, SVM) son algoritmos de clasificación que buscan el **hiperplano de máximo margen** que separa las clases en el espacio de características [?].

Dado un conjunto de entrenamiento $\{(\mathbf{x}_i, y_i)\}_{i=1}^n$ con $y_i \in \{-1, +1\}$, la SVM resuelve el problema de optimización:

$$\min_{\mathbf{w}, b, \xi} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_{i=1}^n \xi_i \quad \text{s.a.} \quad y_i(\mathbf{w}^\top \mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0 \quad (1.9)$$

donde C es el parámetro de regularización que controla la tolerancia al error de clasificación ($C = 1,5$ en este sistema) y ξ son variables de holgura que permiten clasificación blanda.

Kernel**RBF**

Para datos no linealmente separables en el espacio original, la SVM proyecta los datos a un espacio de mayor dimensión mediante la **función de kernel**. El kernel de Base Radial (*Radial Basis Function*, RBF) empleado en este sistema calcula la similitud entre dos puntos como:

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma\|\mathbf{x}_i - \mathbf{x}_j\|^2\right) \quad (1.10)$$

donde γ controla el radio de influencia de cada vector de soporte. El kernel RBF es especialmente adecuado para embeddings de alta dimensión porque puede construir fronteras de decisión no lineales arbitrariamente complejas, adaptándose a la distribución de los casos jurídicos en el espacio semántico.

Ventaja **en** **el** **dominio** **jurídico**

La SVM con kernel RBF maximiza el margen de separación entre las clases, lo que le confiere una robustez natural ante casos límite: quejas que caen en la frontera entre *Competente* y *No Competente* se clasifican según el hiperplano de máximo margen, no según la densidad local de puntos.

1.5.2 Regresión**Logística**

La **Regresión Logística** es un modelo de clasificación lineal que estima la probabilidad de pertenencia a la clase positiva mediante la función logística (sigmoide):

$$P(y = 1 | \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + e^{-(\mathbf{w}^\top \mathbf{x} + b)}} \quad (1.11)$$

Los parámetros $\mathbf{w}$ y b se estiman minimizando la función de pérdida de entropía cruzada binaria:

$$\mathcal{L}(\mathbf{w}, b) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \log \hat{p}_i + (1 - y_i) \log(1 - \hat{p}_i) \right] + \lambda \|\mathbf{w}\|^2 \quad (1.12)$$

donde λ es el término de regularización L2. La configuración empleada en este sistema es `max_iter=1000` con regularización por defecto.

Interpretabilidad. La Regresión Logística produce coeficientes w_j directamente interpretables: un coeficiente positivo asociado a una dimensión del embedding indica que ese componente favorece la clasificación como *Competente*. Esta propiedad es valiosa en el contexto jurídico, donde la explicabilidad de la decisión puede ser requerida institucionalmente.

1.5.3 Ensamble de Árboles — Random Forest

El **Random Forest** es un método de ensamble que construye múltiples árboles de decisión sobre subconjuntos aleatorios de los datos y las características, promediando sus predicciones para reducir la varianza [?]:

$$\hat{y}_{RF}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}) \quad (1.13)$$

donde B es el número de árboles ($B = 200$ en este sistema) y T_b es el b -ésimo árbol entrenado sobre una muestra bootstrap.

Limitación en alta dimensión. Los árboles de decisión particionan el espacio de características con cortes ortogonales a los ejes. En espacios de $d = 768$ dimensiones con distribuciones continuas como los embeddings de Sentence-BERT, esta estrategia de partición es menos eficiente que la del hiperplano de máximo margen de la SVM. Esto explica el menor rendimiento observado del Random Forest frente a SVM y Regresión Logística en los experimentos.

1.6 Estrategias de Entrenamiento y Validación Robusta

La selección y evaluación de los modelos requiere estrategias que garanticen que las métricas reportadas reflejan la capacidad de generalización real del sistema y no son artefactos de una partición particular de los datos.

1.6.1 Validación Cruzada K-Fold Estratificada

La **validación cruzada K-fold** divide el conjunto de datos en K particiones (*folds*) de tamaño aproximadamente igual. En cada iteración, $K - 1$ particiones se usan para entrenamiento y la restante para evaluación, rotando hasta que todos los datos hayan sido usados como conjunto de prueba:

$$\overline{F1}_{CV} = \frac{1}{K} \sum_{k=1}^K F1_k \quad (1.14)$$

La variante **estratificada** garantiza que la proporción de clases se mantiene en cada pliegue, lo cual es crítico ante corpus con posible desbalance entre quejas competentes e incompetentes.

En este proyecto se empleó $K = 5$, lo que significa que en cada iteración el 80 % de los datos se usa para entrenamiento y el 20 % restante para evaluación. La desviación estándar entre pliegues (σ_{CV}) se usa como medida de **estabilidad** del modelo:

$$\sigma_{CV} = \sqrt{\frac{1}{K} \sum_{k=1}^K (F1_k - \overline{F1}_{CV})^2} \quad (1.15)$$

Un σ_{CV} cercano a cero indica que el modelo no es sensible a la partición específica de los datos, condición deseable para un sistema de apoyo en producción.

1.6.2 Evaluación mediante Métricas de Inteligencia

Accuracy

La **exactitud** (*accuracy*) mide la proporción de predicciones correctas sobre el total de instancias:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1.16)$$

En el contexto de la DDP, donde las consecuencias de un falso negativo (queja competente clasificada como incompetente) son más graves que las de un falso positivo, la accuracy por sí sola no es suficiente.

Precisión, **Recall** **y** **F1-Score**

$$\text{Precisión} = \frac{TP}{TP + FP} \quad (1.17)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (1.18)$$

$$F1 = \frac{2 \cdot \text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} \quad (1.19)$$

El **F1 ponderado** (*weighted F1*) extiende este cálculo a ambas clases, ponderando por el número de instancias de cada una:

$$F1_w = \sum_{c \in C} \frac{n_c}{N} \cdot F1_c \quad (1.20)$$

donde n_c es el número de instancias de la clase c y N el total. Esta métrica es la principal para la selección del modelo óptimo en este sistema.

Análisis **de** **la** **varianza** σ

La desviación estándar del F1 en validación cruzada (σ_{CV}) complementa el valor medio al revelar la estabilidad del modelo. Un modelo con $\overline{F1}_{CV} = 0,98$ y $\sigma_{CV} = 0,04$ puede ser menos confiable que uno con $\overline{F1}_{CV} = 0,97$ y $\sigma_{CV} = 0,00$, pues el primero es sensible a la distribución particular de los datos en cada pliegue.

1.6.3 Carga Perezosa y Serialización con joblib

La **carga perezosa** (*lazy loading*) es un patrón de diseño donde los recursos computacionalmente costosos —como la carga del modelo de Sentence-BERT o el modelo de spaCy— se inicializan

únicamente en el momento en que son requeridos por primera vez, no al inicio del programa. Esto reduce el tiempo de arranque del sistema y el consumo de memoria en instancias donde solo se requiere predicción.

La **serialización** de los pipelines se realiza mediante la biblioteca `joblib`, que convierte el objeto Python completo —vectorizador y clasificador juntos— en un archivo binario `.pkl`:

Pipeline(vectorizador, clasificador) $\xrightarrow{\text{joblib.dump}}$ `modelo.pkl` $\xrightarrow{\text{joblib.load}}$ Pipeline listo para inferencia

Esta estrategia es técnicamente relevante por tres razones:

1. **Integridad:** Vectorizador y clasificador se serializan juntos, eliminando el riesgo de usar un clasificador entrenado con un vectorizador distinto al de producción.
2. **Eficiencia:** `joblib` emplea compresión paralela y mapeo de memoria (*memory mapping*), resultando significativamente más rápido que `pickle` estándar para arrays numéricos de gran tamaño como los parámetros internos de Sentence-BERT.
3. **Reproducibilidad:** El archivo `.pkl` incluye la semilla aleatoria (`random_state=42`) y todos los hiperparámetros, garantizando que las predicciones son idénticas entre sesiones y entornos de ejecución.

La convención de nomenclatura de los archivos serializados es:

`<vectorizador>__<clasificador>__<timestamp>.pkl`

lo que permite trazabilidad temporal de los modelos entrenados y facilita la comparación y auditoría de versiones en el contexto institucional de la DDP.

1.7 Fundamentos de arquitectura de software

1.7.1 Definición de arquitectura de software

La arquitectura de software es el conjunto de decisiones fundamentales que definen la estructura, organización y comportamiento de un sistema. Al utilizar patrones y principios de diseño, establece un marco de referencia para la construcción, evolución y mantenimiento del software.

Más allá de organizar componentes, la arquitectura determina cómo se gestionan las interacciones para cumplir con los atributos de calidad (escalabilidad, seguridad, desempeño) y los requerimientos funcionales y no funcionales. Constituye el puente entre los objetivos organizacionales y las soluciones tecnológicas, siendo esencial para garantizar la integridad y viabilidad del sistema [?].

1.7.2 Importancia en el Desarrollo de Sistemas

La arquitectura es el cimiento técnico que asegura la correcta construcción del sistema adecuado para el negocio, destacando por:

- **Gestión de la Complejidad:** Coordina procesos y dependencias mediante flexibilidad y adaptación, controlando factores desconocidos o cambios no anticipados.
- **Mitigación de Riesgos Tempranos:** Permite un enfoque proactivo para detectar y neutralizar vulnerabilidades antes de que afecten la operación.
- **Escalabilidad:** Garantiza la adaptación del sistema ante variaciones en la carga de trabajo sin afectar su rendimiento ni requerir reingeniería.
- **Mantenibilidad:** Facilita modificaciones, correcciones y mejoras futuras gracias a un diseño organizado y claro.
- **Toma de Decisiones:** Establece el marco para la elección de tecnologías y la gestión de compromisos técnicos (*trade-offs*).

1.7.3 Factores de calidad

Basados en modelos internacionales (ISO/IEC 25010), los atributos de calidad evalúan y garantizan la viabilidad a largo plazo de la arquitectura elegida [?]. En un sistema de gestión de denuncias, destacan:

- **Rendimiento y Eficiencia:** Optimización de recursos y latencias mínimas en la comunicación entre microservicios, asegurando una experiencia fluida al cargar evidencias [?].
- **Mantenibilidad:** Facilidad para actualizar o corregir el sistema. El uso de *Docker* y el patrón de tres capas permite modificar microservicios independientemente sin afectar la estabilidad [?].

- **Escalabilidad:** Capacidad de escalar horizontalmente servicios críticos ante altas demandas de usuarios, optimizando la infraestructura institucional [?].
- **Seguridad y Confidencialidad:** Protección de la identidad del denunciante e integridad del expediente mediante cifrado de datos y control de acceso (RBAC).
- **Disponibilidad y Tolerancia a Fallos:** Aislamiento de errores entre servicios (ej. la caída del módulo analítico no afecta el sistema central), previniendo el colapso total.

Estos factores aseguran que la plataforma sea una herramienta robusta, confiable y duradera para la Defensoría.

1.8 Arquitectura del Sistema

1.8.1 Visión general de la arquitectura híbrida

Para el desarrollo de la “Plataforma Web para la Denuncia Segura y Prevención de Riesgos Digitales”, se ha seleccionado una arquitectura de microservicios, la cual se complementa internamente con un patrón de tres capas para la organización de cada servicio. Esta decisión técnica no es arbitraria; se fundamenta en la necesidad de superar las limitaciones de las arquitecturas monolíticas tradicionales, donde el alto acoplamiento entre componentes suele derivar en dificultades para el mantenimiento y la escalabilidad ante una demanda creciente [?].

Al adoptar un enfoque de microservicios, el sistema se descompone en unidades funcionales autónomas que se comunican a través de protocolos ligeros. Según Villamizar et al., esta estructura permite que cada componente evolucione de manera independiente, facilitando la innovación tecnológica y permitiendo que fallos en un servicio específico (como el procesador de lenguaje natural en Python) no comprometan la disponibilidad global de la plataforma administrativa en Java [?].

1.8.2 Arquitectura de Microservicios

La arquitectura de microservicios se define como un enfoque de desarrollo de software donde una aplicación se descompone en un conjunto de servicios pequeños, autónomos y altamente cohesivos. Cada uno de estos componentes se ejecuta como un proceso independiente y se comunica con los

demás a través de mecanismos ligeros, generalmente interfaces de programación de aplicaciones (APIs) basadas en el protocolo HTTP.

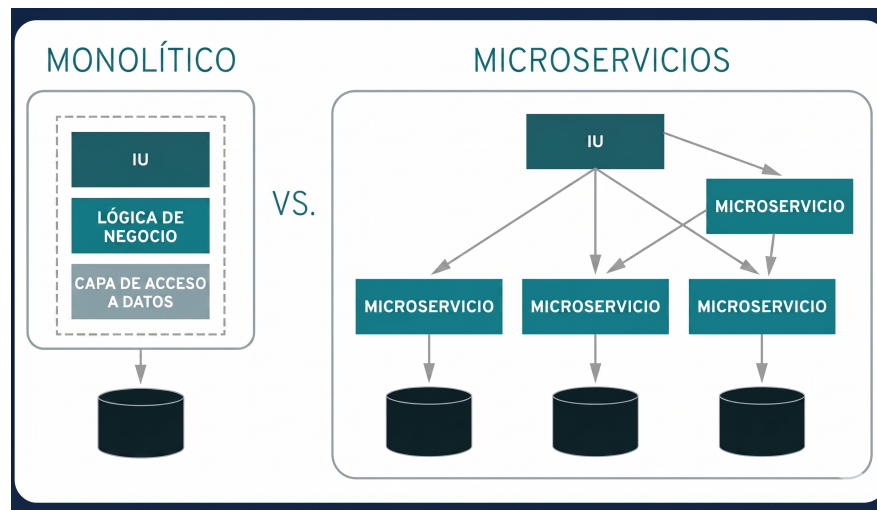


Figura 1.1: Comunicación entre microservicios.

A diferencia de las arquitecturas monolíticas, donde una falla en un módulo puede comprometer la totalidad del sistema, los microservicios permiten que cada unidad funcional se enfoque en una capacidad específica del negocio, facilitando su desarrollo, despliegue y escalado de forma totalmente autónoma [?]. **Ventajas de su uso:**

- **Aislamiento y Resiliencia ante Fallos:** Dado que cada servicio opera en un entorno de ejecución separado, un error crítico o una excepción no controlada en un módulo específico no provoca un efecto dominó sobre el resto de la plataforma. Esta característica es vital para mantener la alta disponibilidad requerida en entornos institucionales [?].
- **Escalabilidad Granular y Eficiente:** Permite realizar un escalado horizontal selectivo. En lugar de replicar la aplicación completa, es posible asignar recursos de infraestructura adicionales únicamente a aquellos servicios que experimentan una alta demanda de procesamiento, optimizando significativamente el uso del hardware y los costos operativos [?] [?].
- **Heterogeneidad Tecnológica:** Facilita el uso de diferentes lenguajes de programación y bases de datos según las necesidades específicas de cada módulo. En este proyecto, dicha flexibilidad permite la coexistencia de servicios robustos en Java con módulos especializados en análisis de datos desarrollados en Python [?].

- **Agilidad en el Despliegue Continuo:** Al reducir el acoplamiento entre funciones, se acortan los ciclos de desarrollo y se permite la actualización constante de módulos individuales sin necesidad de reiniciar o interrumpir la operación global del sistema [?].

Justificación para la Defensoría de los Derechos Politécnicos: La elección de este enfoque para la plataforma de denuncia segura responde a la complejidad intrínseca de los procesos administrativos y legales del Instituto. El sistema requiere gestionar múltiples roles y flujos de trabajo (denunciantes, abogados, administradores) que poseen demandas computacionales distintas. Se adoptó esta arquitectura para asegurar la continuidad operativa: si el microservicio encargado del procesamiento de lenguaje natural llegara a saturarse, los módulos de registro de quejas y gestión de expedientes continuarían funcionando de forma ininterrumpida.

1.8.3 Arquitectura interna de los servicios: Patrón de 3 capas

La arquitectura de tres niveles (o capas) es un modelo de diseño de software cuya finalidad primordial es la separación de las responsabilidades y el desacoplamiento de las etapas de procesamiento de una aplicación. Esta estructura organiza el software en tres niveles lógicos y físicos claramente diferenciados: el nivel de presentación, el nivel de aplicación (o lógica de negocio) y el nivel de datos [?] [?].

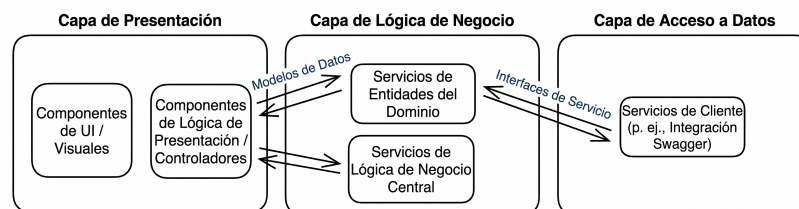


Figura 1.2: Patrón de Diseño de 3 capas.

En este proyecto, dicha estructura conforma de manera exacta el diseño interno de cada microservicio, garantizando que la lógica institucional esté aislada de las interfaces de usuario y de los motores de persistencia.

- **Nivel de Presentación (Interfaz de Usuario):** Es la capa más externa del servicio, encargada de la interacción directa con el usuario o con otros servicios externos. Su función principal es presentar la información y recolectar datos de entrada para enviarlos a la capa de aplicación.

Al estar desacoplada, permite realizar cambios estéticos o funcionales en la interfaz (Angular) sin alterar las reglas de negocio subyacentes [?] [?].

- **Nivel de Aplicación (Lógica de Negocio):** Considerado el núcleo del servicio, es donde se modelan las reglas del negocio y se toman las decisiones lógicas. Según Coti Colop, la ubicación de las reglas en esta capa central es crítica para asegurar la funcionalidad y la integridad del sistema, ya que actúa como un puente que coordina las peticiones de la interfaz y la gestión de los datos, protegiendo la coherencia de los procesos legales y administrativos de la Defensoría [?].
- **Nivel de Datos (Persistencia):** Es la capa responsable del almacenamiento y la recuperación de la información. Gestiona la comunicación con el Sistema de Gestión de Bases de Datos (PostgreSQL), permitiendo que la persistencia sea transparente para las capas superiores. Esto significa que el motor de base de datos podría ser optimizado o sustituido con un impacto mínimo en la lógica de negocio [?].

Ventajas de su uso:

- **Independencia y Escalabilidad:** Cada nivel puede ser escalado de forma independiente según la demanda específica. Por ejemplo, si el procesamiento de datos aumenta, se puede fortalecer la capa de aplicación sin necesidad de modificar la interfaz de usuario [?] [?].
- **Mantenibilidad y Flexibilidad:** Facilita el mantenimiento a largo plazo al permitir que cada capa sea desarrollada, probada y actualizada simultáneamente por distintos integrantes del equipo, lo que reduce significativamente el tiempo total de desarrollo y el riesgo de introducir errores colaterales [?].
- **Confiabilidad Operativa:** Al operar de manera aislada, una interrupción o fallo en la capa de presentación no compromete la integridad de los datos ni la ejecución de los procesos de negocio en curso, aumentando la resiliencia del microservicio [?].

1.8.4 Servidor web y proxy inverso (Nginx)

NGINX es una infraestructura de software de código abierto diseñada para el servicio de contenido web, el funcionamiento como proxy inverso y la gestión de balanceo de carga. A diferencia de los servidores tradicionales basados en hilos, NGINX destaca por su arquitectura asíncrona basada en eventos, la cual permite gestionar un alto volumen de conexiones concurrentes con una utilización mínima de memoria y CPU.

En el ecosistema de este proyecto, NGINX actúa como el punto de entrada unificado, gestionando el flujo de datos entre el cliente y los servicios internos.

Ventajas de su uso:

- **Rendimiento y Manejo de Concurrencia:** Gracias a su arquitectura no bloqueante, el servidor puede procesar miles de peticiones simultáneas sin los costos asociados a la creación de nuevos procesos por cada conexión.
- **Balanceo de Carga Eficiente:** Como proxy inverso, NGINX distribuye de manera inteligente las peticiones entrantes entre las distintas instancias de los microservicios.
- **Caché y Optimización de Contenido:** Posee la capacidad de almacenar temporalmente contenido estático y respuestas frecuentes, reduciendo el tiempo de respuesta percibido.
- **Terminación de SSL/TLS:** NGINX permite centralizar la gestión de certificados de seguridad y el cifrado de las comunicaciones en un solo punto.

Implementación como API Gateway: Dentro de una arquitectura de microservicios, exponer cada módulo directamente a la red pública representaría una vulnerabilidad crítica. En este proyecto, NGINX se implementa bajo el patrón de *API Gateway* (puerta de enlace), interceptando todas las peticiones y enrutándolas mediante reglas de despacho hacia el microservicio interno correspondiente.

1.9 Tecnologías de Desarrollo Backend

1.9.1 Lenguaje de programación: Java

Java es un lenguaje de programación de propósito general, orientado a objetos y una plataforma informática de amplia adopción en el ámbito empresarial. Se destaca históricamente por su portabilidad, seguridad y fiabilidad, siendo una de las tecnologías preferidas para el desarrollo de sistemas distribuidos y aplicaciones de misión crítica.

Seguridad y tipado estricto

La seguridad en Java no es una capa superficial, sino una característica intrínseca de su diseño. Se basa en el modelo de *Sandbox* (Caja de Arena), que restringe la ejecución de código no confiable.

Java implementa un sistema de tipado fuerte que detecta errores en tiempo de compilación, lo que reduce drásticamente las vulnerabilidades comunes.

Arquitectura de la Java Virtual Machine (JVM)

La JVM es el corazón técnico que permite la independencia de plataforma mediante una capa de abstracción entre el código fuente y el hardware. La compilación JIT (Just-In-Time) analiza las secciones de código que se ejecutan con mayor frecuencia y las traduce directamente a código máquina optimizado.

Gestión de memoria: Garbage Collection (GC)

A diferencia de lenguajes como C++, Java gestiona la memoria de forma automática, eliminando el riesgo de fugas de memoria. El recolector de basura divide la memoria *Heap* en regiones: *Young Generation* y *Old Generation*, utilizando el algoritmo Mark-and-Sweep.

Multihilo y concurrencia

Para manejar múltiples denuncias simultáneas, Java utiliza un modelo de hilos nativos gestionados por el sistema operativo pero controlados por la JVM. Spring Boot utiliza *Thread Pools* que mantienen un número N de hilos activos listos para procesar tareas de una cola de espera.

1.9.2 Framework: Spring Boot

Spring Boot es una extensión del ecosistema Spring diseñada para simplificar el proceso de configuración y despliegue de aplicaciones de grado empresarial. Se basa en el principio de “*Convention over Configuration*” (convención sobre configuración), lo que permite crear microservicios autónomos y listos para producción con un esfuerzo de configuración mínimo [?].

Pilares fundamentales

- **Configuración Automática:** El framework analiza las dependencias presentes en el *classpath* y configura automáticamente los componentes.

- **Spring Starters:** Paquetes de dependencias predefinidos que agrupan todas las librerías necesarias para una funcionalidad específica.
- **Aplicaciones Standalone:** Integra servidores embebidos (Tomcat, Jetty, Undertow) generando un archivo JAR ejecutable.

Inversión de Control (IoC) e Inyección de Dependencias

El núcleo de Spring Boot se basa en el contenedor de Inversión de Control (IoC) y el patrón de Inyección de Dependencias (DI). Spring gestiona el ciclo de vida de los objetos (*Beans*) e inyecta las dependencias necesarias en tiempo de ejecución.

Spring Boot Actuator

Spring Boot Actuator es un subproyecto del ecosistema Spring que proporciona funcionalidades críticas para la *observabilidad* y el monitoreo de aplicaciones en entornos de producción. Según [?], esta herramienta permite obtener información detallada sobre el estado de ejecución de los componentes internos de la aplicación sin necesidad de implementar código adicional para la recolección de métricas.

Una vez habilitado, el framework expone una serie de puntos de acceso o *endpoints* bajo la ruta base `/actuator`. Por defecto, y por razones de seguridad, solo el *endpoint* de salud (`/health`) se encuentra activo, devolviendo un estado simplificado como `{"status": "UP"}` para confirmar que la aplicación está operativa.

Endpoints Principales y Gestión de Beans Para un monitoreo exhaustivo, es posible habilitar la exposición de otros puntos de acceso mediante la propiedad `management.endpoints.web.exposure.include` en el archivo de configuración. Los *endpoints* más relevantes incluyen:

- **/health:** Informa sobre el estado de la aplicación y sus dependencias.
- **/metrics:** Expone métricas de rendimiento como el uso de memoria JVM y CPU.
- **/loggers:** Permite visualizar y modificar los niveles de log en tiempo de ejecución.
- **/beans:** Muestra una estructura JSON detallada de todos los *Spring Beans* registrados.

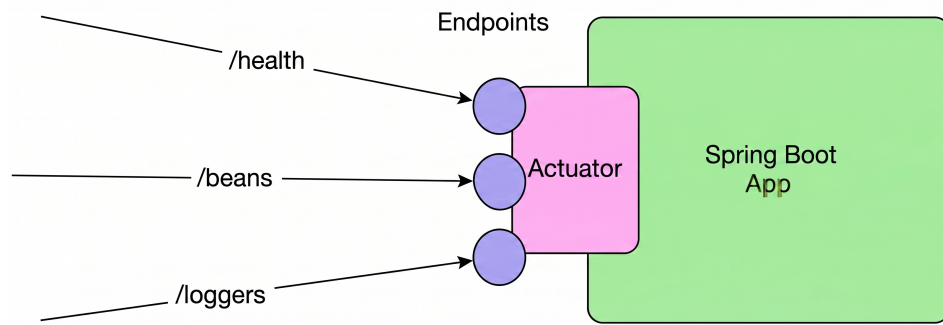


Figura 1.3: Visualización de los Endpoints de monitoreo proporcionados por Spring Boot Actuator.[?]

Esta capacidad de inspección es fundamental para asegurar la integridad de la inversión de control, permitiendo verificar que componentes críticos como los repositorios de persistencia han sido correctamente instanciados [?].

Importancia en Entornos de Contenedores En arquitecturas modernas basadas en Docker y Kubernetes, Actuator facilita la implementación de pruebas de vida y disponibilidad (*Liveness* y *Readiness probes*), optimizando la disponibilidad del servicio mediante la gestión automatizada del orquestador.

Spring Security (JWT y OAuth2)

Spring Security proporciona un soporte integral para la implementación de servidores de recursos (*Resource Servers*) bajo el estándar OAuth 2.0, utilizando tokens **JWT (JSON Web Tokens)** para la autorización de solicitudes [?]. Este esquema permite que la aplicación valide de forma segura la identidad y los permisos del cliente mediante el uso de tokens portadores (*Bearer Tokens*) sin necesidad de mantener sesiones en el servidor.

Dependencias Mínimas para JWT Para que un servidor de recursos sea funcional y capaz de decodificar y verificar firmas criptográficas, es necesaria la inclusión de dos bibliotecas fundamentales en el archivo `pom.xml` [?]:

- `spring-security-oauth2-resource-server`: Contiene el soporte base para la gestión de servidores de recursos.

- **spring-security-oauth2-jose**: Proporciona la infraestructura necesaria para el procesamiento de objetos *Javascript Object Signing and Encryption* (JOSE), requerida para validar los JWT.

Al establecer esta propiedad, el sistema lanza un proceso de descubrimiento determinista que consulta los *metadata endpoints* del proveedor para configurar las estrategias de validación de firmas y algoritmos de confianza (por defecto RS256) [?].

Arquitectura y Proceso de Autenticación El componente central es el `JwtAuthenticationProvider`, el cual utiliza un `JwtDecoder` para procesar el token y un `JwtAuthenticationConverter` para transformar los *claims* del JSON en autoridades de seguridad [?].

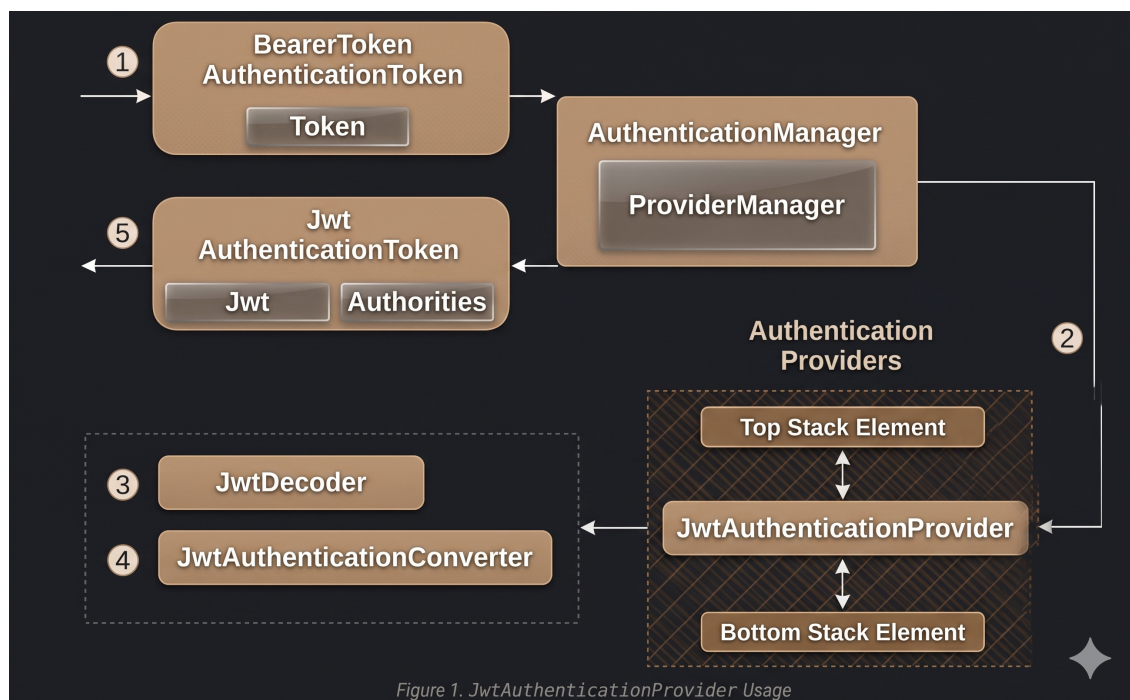


Figura 1.4: Uso de `JwtAuthenticationProvider` en el flujo de autenticación de Spring Security [?].

Una vez que el servidor de recursos recibe una petición con el encabezado **Authorization: Bearer**, se ejecutan las siguientes validaciones automáticas [?]:

1. **Firma:** Se verifica contra la llave pública obtenida del `jwks_url`.
2. **Vigencia:** Se validan las marcas de tiempo de expiración (*exp*) y no antes de (*nbf*).
3. **Emisor:** Se confirma que el *claim iss* coincida con el configurado en la aplicación.

Autorización Basada en Scopes Los tokens emitidos suelen incluir un atributo `scope` o `scp`. Spring Security mapea estos atributos prefijándolos con la cadena `SCOPE_`, permitiendo restringir el acceso a rutas específicas mediante la cadena de filtros [?]:

@Bean

```
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .authorizeHttpRequests((authorize) -> authorize
            .requestMatchers("/reportes/**").access(hasScope("reportes:lectura"))
            .anyRequest().authenticated()
        )
        .oauth2ResourceServer((oauth2) -> oauth2.jwt(Customizer.withDefaults()));
    return http.build();
}
```

Este mecanismo garantiza que el sistema solo procese solicitudes de usuarios que posean los permisos exactos requeridos para interactuar con los expedientes de la Defensoría [?].

Spring Cloud (Config Server y Eureka Service Discovery)

En el desarrollo de arquitecturas distribuidas para la Defensoría de los Derechos Politécnicos, la gestión manual de configuraciones y la localización de servicios resulta ineficiente. Spring Cloud proporciona una serie de herramientas para resolver problemas comunes en sistemas distribuidos, tales como la gestión de configuraciones y el descubrimiento de servicios [?].

Spring Cloud Config Server Este componente actúa como un servidor centralizado para la gestión de configuraciones externas en un sistema distribuido. Sus características principales incluyen [?]:

- **Centralización:** Permite desacoplar las propiedades de configuración (como credenciales de bases de datos o parámetros del clasificador de IA) del código fuente, almacenándolas en un repositorio centralizado (generalmente Git) [?].
- **Gestión por Entornos:** Facilita el manejo de diferentes perfiles (desarrollo, pruebas y producción) de forma transparente para los microservicios [?].

- **Actualización Dinámica:** A través de anotaciones como `@RefreshScope`, el sistema puede actualizar parámetros en tiempo de ejecución sin necesidad de reiniciar las instancias de los servicios, garantizando la continuidad operativa [?].

Eureka Service Discovery Dada la naturaleza dinámica de los contenedores Docker en el entorno del proyecto, donde las direcciones IP pueden cambiar frecuentemente, se implementa Eureka como mecanismo de localización automática [?]:

- **Eureka Server:** Funciona como un directorio de servicios donde cada microservicio se registra al iniciar, informando su dirección de red y estado de salud [?].
- **Eureka Client:** Permite que los microservicios se localicen entre sí mediante nombres lógicos (ej. `servicio-ia-clasificador`) en lugar de direcciones IP estáticas, facilitando el balanceo de carga y la tolerancia a fallos [?].
- **Monitoreo de Estado:** El servidor Eureka utiliza un mecanismo de *heartbeat* para detectar si una instancia ha caído, eliminándola automáticamente del registro para evitar errores de comunicación en el ecosistema [?].

Esta infraestructura asegura que el sistema sea escalable y mantenible, permitiendo una comunicación fluida entre el repositorio digital y el módulo de consulta ciudadana [?].

1.9.3 Interfaces de Programación (API REST): Estándares de interoperabilidad, documentación y versionamiento

Las interfaces de programación de aplicaciones bajo el estilo arquitectónico REST (*Representational State Transfer*) constituyen el estándar de comunicación entre los microservicios del sistema y el *front-end*. Este enfoque permite la interoperabilidad entre plataformas heterogéneas mediante el uso de protocolos estándar de la web [?].

Estándares de Interoperabilidad y HTTP/2 Para garantizar una comunicación eficiente y escalable, el sistema se adhiere a los principios de REST, utilizando los verbos de transferencia HTTP (GET, POST, PUT, DELETE) para definir operaciones semánticas sobre los recursos. La implementación aprovecha las ventajas de **HTTP/2**, que introduce la multiplexación de peticiones y la compresión de cabeceras, reduciendo significativamente la latencia en la carga de expedientes y reportes dentro de la plataforma [?].

Estrategias de Versionamiento y Rate Limiting Dado que el sistema está sujeto a evoluciones normativas y técnicas, se implementan mecanismos de control para asegurar la compatibilidad:

- **Versioning:** Se adopta el versionado a través de la URI (ej. /v1/expedientes), lo que permite coexistir diferentes versiones de un mismo servicio mientras se migran las funcionalidades [?].
- **Rate Limiting (Bucket4j):** Para proteger la integridad del repositorio digital ante posibles ataques de denegación de servicio o saturación, se implementa un control de tasa de peticiones. Esto asegura que el consumo de recursos se mantenga dentro de los límites operativos permitidos por la infraestructura [?].

Esta capa de servicios garantiza que la información de los derechos politécnicos sea accesible, segura y fácil de integrar con futuros módulos institucionales [?].

1.10 Seguridad y Comunicación de Datos (En Tránsito)

La seguridad de los datos en tránsito se refiere a las medidas de protección aplicadas a la información mientras esta se desplaza a través de una red, ya sea pública o privada. En una arquitectura de microservicios, asegurar el canal de comunicación es crítico para evitar ataques de intermediario (*Man-in-the-Middle*) y garantizar la integridad de los expedientes de la Defensoría[?].

1.10.1 Protocolos de Transporte Seguro: TLS 1.3

Para cifrar las comunicaciones en la capa de transporte, el sistema implementa el protocolo **TLS (Transport Layer Security) 1.3**[?]. Esta versión ofrece mejoras significativas sobre sus predecesores:

- **Reducción de Latencia:** Optimiza el proceso de *handshake*, permitiendo que la conexión se establezca más rápido, lo cual es vital para el rendimiento de los microservicios[?].
- **Seguridad Reforzada:** Elimina algoritmos criptográficos obsoletos y vulnerables, obligando al uso de técnicas modernas de cifrado autenticado[?].
- **Privacidad de Forwarding:** Garantiza que, incluso si una clave de sesión se ve comprometida en el futuro, las comunicaciones pasadas permanezcan cifradas[?].

Mecanismos de Seguridad Reforzada A diferencia de sus predecesores, TLS 1.3 elimina el soporte para algoritmos criptográficos considerados obsoletos o inseguros (como MD5, SHA-1 y suites de cifrado de exportación), obligando al uso exclusivo de técnicas modernas de cifrado autenticado basadas en **AEAD** (*Authenticated Encryption with Associated Data*) [?, ?]. Esto garantiza que cada paquete de información no solo esté cifrado, sino que su autenticidad sea verificada de forma simultánea, impidiendo ataques de manipulación de datos [?].

Optimización del Handshake y 0-RTT Una de las mejoras más significativas de TLS 1.3 es la reducción drástica en la latencia de conexión [?]:

- **Handshake de un solo viaje (1-RTT):** Reduce a la mitad el número de interacciones necesarias entre el cliente y el servidor para establecer una clave de cifrado común, en comparación con el proceso requerido en TLS 1.2 [?].
- **0-RTT (Zero Round Trip Time Resumption):** Permite que los usuarios que han establecido una conexión previa puedan enviar datos cifrados en el primer mensaje de la reconexión, acelerando la experiencia de usuario en el módulo de consulta ciudadana [?].

Perfect Forward Secrecy (PFS) Al hacer obligatorio el intercambio de claves mediante *Ephemeral Diffie-Hellman*, TLS 1.3 asegura que, ante un hipotético compromiso de la clave privada del servidor en el futuro, las sesiones de comunicación capturadas en el pasado permanezcan cifradas e inaccesibles, proporcionando protección a largo plazo para los expedientes digitales [?].

Esta arquitectura híbrida asegura que los procesos pesados, como el análisis del clasificador neuro-simbólico, no degraden la respuesta del sistema frente al ciudadano [?].

1.10.2 Resiliencia del Sistema: Patrones API Gateway (Spring Cloud Gateway) y Circuit Breaker (Resilience4j)

En una arquitectura distribuida como la propuesta para la Defensoría de los Derechos Politécnicos, la resiliencia se define como la capacidad del sistema para recuperar su funcionamiento normal tras un fallo o para seguir operando ante errores parciales [?]. Para evitar que un fallo en un microservicio sature los recursos de toda la infraestructura, se implementan patrones de diseño de alta disponibilidad [?].

API Gateway con Spring Cloud Gateway El patrón API Gateway actúa como un punto de entrada único y centralizado para todas las solicitudes de los clientes, gestionando la complejidad del ecosistema de microservicios de forma transparente [?]. Sus funciones críticas incluyen:

- **Enrutamiento Dinámico:** Dirige las peticiones entrantes hacia el microservicio correspondiente mediante la resolución de nombres proporcionada por Eureka [?].
- **Seguridad Perimetral:** Centraliza la validación de tokens JWT y la aplicación de políticas de seguridad antes de que la petición alcance los servicios internos [?].
- **Abstracción de Red:** Permite que el *front-end* interactúe con un único dominio, ocultando la topología de la red interna y simplificando el manejo de CORS (*Cross-Origin Resource Sharing*) [?].

Patrón Circuit Breaker mediante Resilience4j El patrón de interruptor o *Circuit Breaker* es fundamental para prevenir fallos en cascada dentro del sistema [?]. Utilizando la librería **Resilience4j**, el sistema monitoriza las llamadas a servicios externos (como el repositorio digital o el clasificador de IA) y gestiona su estado de la siguiente manera [?]:

- **Estado Cerrado (Closed):** El flujo de peticiones es normal. El interruptor contabiliza los éxitos y fallos de las operaciones de red [?].
- **Estado Abierto (Open):** Si el umbral de fallos se supera (debido a tiempos de espera agotados o errores de servidor), el interruptor se abre. En este estado, todas las peticiones fallan de inmediato, protegiendo los recursos de la infraestructura [?].
- **Estado Entreabierto (Half-Open):** Tras un periodo de espera, se permite el paso de un número limitado de peticiones de prueba para verificar si el servicio remoto se ha recuperado antes de volver al estado cerrado [?].

Estrategias de Fallback Complementando al *Circuit Breaker*, se definen métodos de *fallback* que proporcionan una respuesta alternativa predeterminada cuando un servicio no está disponible [?]. Esto asegura que el ciudadano siempre reciba una respuesta informativa o una versión simplificada del servicio en lugar de un error crítico del sistema, manteniendo la confianza en la plataforma institucional [?].

1.11 Infraestructura y Orquestación

La infraestructura moderna para el despliegue de microservicios se fundamenta en la abstracción del hardware y la automatización del entorno de ejecución. Para el sistema de la Defensoría, se adopta un enfoque basado en contenedores, lo que garantiza que la aplicación se comporte de manera idéntica desde el entorno de desarrollo local hasta el servidor de producción en Hostinger [?].

1.11.1 Contenerización: Fundamentos de Docker, Dockerfile y gestión con Docker Compose

Docker es la plataforma de virtualización a nivel de sistema operativo que permite empaquetar software en unidades estandarizadas llamadas contenedores [?]. Esta tecnología es crítica para asegurar la portabilidad del repositorio digital y sus microservicios [?].

Componentes y Definición El proceso de contenerización se rige por los siguientes elementos técnicos [?]:

- **Dockerfile:** Es un archivo de configuración basado en texto que contiene todas las instrucciones necesarias para construir una imagen de Docker [?]. En este proyecto, se utiliza para definir el entorno de ejecución de Java, copiar los archivos JAR generados por Maven y configurar las variables de entorno necesarias [?].
- **Imágenes y Contenedores:** Una imagen es una plantilla de solo lectura que incluye el código, las librerías y las dependencias [?]. El contenedor es la instancia ejecutable de dicha imagen, operando de forma aislada para evitar conflictos de librerías en el servidor [?].
- **Docker Compose:** Es una herramienta para definir y ejecutar aplicaciones Docker de múltiples contenedores [?]. Mediante un archivo `docker-compose.yml`, se orquestan servicios como la base de datos PostgreSQL, el servidor Eureka y los microservicios de Spring Boot, permitiendo levantar toda la infraestructura con un único comando [?].

1.11.2 Orquestación de Contenedores: Kubernetes (K8s), gestión de Pods, Deployments y escalabilidad horizontal (HPA)

Kubernetes es una plataforma de código abierto diseñada para automatizar el despliegue, el escalado y la gestión de aplicaciones contenerizadas [?]. En este Trabajo Terminal, K8s proporciona la capa de orquestación necesaria para garantizar la alta disponibilidad [?].

Abstracciones de Kubernetes El sistema se organiza mediante los siguientes objetos de orquestación [?]:

- **Pods:** Es la unidad mínima de ejecución en Kubernetes [?]. Un Pod agrupa uno o más contenedores (como el microservicio de Java) que comparten almacenamiento y red, actuando como una única entidad lógica dentro del clúster [?].
- **Deployments:** Describen el estado deseado de los Pods [?]. El controlador de Deployment se encarga de cambiar el estado actual al deseado a una tasa controlada, permitiendo actualizaciones sin tiempo de inactividad (*rolling updates*) y recuperaciones automáticas ante fallos [?].
- **Horizontal Pod Autoscaler (HPA):** Es el mecanismo encargado de la escalabilidad horizontal [?]. El HPA escala automáticamente el número de Pods en un Deployment basándose en métricas de utilización de CPU o memoria [?]. Por ejemplo, si el clasificador neuro-simbólico detecta una carga inusual durante el procesamiento de quejas, el HPA instanciará nuevos Pods para distribuir la carga y mantener la respuesta del sistema [?].

Esta infraestructura de orquestación asegura que la plataforma de la Defensoría sea resiliente y capaz de adaptarse a la demanda de los usuarios de manera autónoma [?].

1.12 Gestión de Bases de Datos

1.12.1 PostgreSQL

PostgreSQL es un sistema de gestión de bases de datos relacionales de objetos (ORDBMS) de código abierto, reconocido por su fiabilidad, robustez de funciones y rendimiento, lo que permite gestionar cargas de trabajo de datos complejas de forma segura [?].

Ventajas de su uso:

- **Integridad y Seguridad de Datos:** Cumple estrictamente con el modelo ACID, garantizando que todas las transacciones se procesen de forma fiable [?].
- **Extensibilidad y Versatilidad:** Permite definir tipos de datos personalizados y soporta diversos lenguajes [?].
- **Soporte y Estándares:** Su alta adherencia a SQL asegura la compatibilidad con múltiples herramientas de análisis [?].

1.13 Persistencia y Gestión del Repositorio Digital (En Reposo)

La persistencia de la información en el sistema de la Defensoría se fundamenta en el uso de bases de datos relacionales de objetos de alto rendimiento. Se define como datos .^{en} reposo.^a toda la información que se encuentra almacenada de forma persistente en soportes físicos, requiriendo mecanismos específicos de integridad y seguridad para su preservación a largo plazo [?]. El repositorio digital no solo actúa como un almacén, sino como un sistema de gestión de activos digitales que garantiza la disponibilidad y la inmutabilidad de los expedientes jurídicos [?].

1.13.1 PostgreSQL Avanzado: Manejo de datos semi-estructurados (JSONB) e indexación GIN para búsqueda de texto completo (Full-text search)

PostgreSQL se ha consolidado como la opción preferida para el repositorio digital debido a su arquitectura extensible y su capacidad para manejar esquemas híbridos que combinan el rigor relacional con la flexibilidad documental [?].

Manejo de JSONB y Modelado NoSQL A diferencia del tipo de datos JSON estándar, que almacena una copia exacta del texto, el formato **JSONB** (*JSON Binary*) almacena los datos en un formato binario descompuesto [?]. Esta implementación permite:

- **Optimización del Rendimiento:** Aunque la inserción conlleva un costo computacional adicional debido al procesamiento binario, el análisis y la recuperación son significativamente más veloces, ya que se evita el re-parseo del texto en cada consulta [?].

- **Evolución del Esquema:** Facilita el almacenamiento de metadatos variables de los expedientes, tales como anexos técnicos o formularios dinámicos, que no se ajustan a una estructura tabular rígida, sin comprometer la capacidad de realizar consultas complejas [?].
- **Operadores de Contención:** Permite el uso de operadores especializados (como @>) para verificar si un documento JSONB contiene ciertos pares clave-valor de forma eficiente [?].

Búsqueda de Texto Completo e Índices GIN Para la recuperación eficiente de información dentro del cuerpo de los documentos legales y quejas, se implementa *Full-Text Search* mediante [?]:

- **Componentes Lingüísticos (tsvector y tsquery):** Los documentos se procesan mediante diccionarios para eliminar *stop-words* y realizar *stemming* (reducción a la raíz), transformándolos en vectores de lexemas que permiten búsquedas semánticas y no solo literales [?].
- **Índices GIN (Generalized Inverted Index):** Este tipo de indexación es fundamental para el manejo de JSONB y búsqueda de texto, ya que funciona como un índice invertido que mapea cada término o clave a la lista de registros que lo contienen, acelerando exponencialmente las consultas sobre los grandes volúmenes de datos que genera la Defensoría [?].

1.13.2 Seguridad y Cifrado en Reposo: Implementación de pgcrypto (AES-256) y validación de integridad mediante hashes SHA-512

La protección de los datos en reposo es una exigencia legal estricta bajo la LFPDPPP para salvaguardar la privacidad de los miembros de la comunidad politécnica [?]. Se utiliza la extensión **pgcrypto** para implementar criptografía robusta a nivel de motor de base de datos [?].

Cifrado Simétrico con AES-256 Se utiliza el estándar **AES (Advanced Encryption Standard)** con llaves de 256 bits en modo CBC o GCM para cifrar campos sensibles como nombres de involucrados, folios internos y descripciones detalladas de vulneraciones de derechos [?]. Este mecanismo asegura que, ante una extracción no autorizada de los archivos físicos de la base de datos (*tablespaces*), la información permanezca cifrada e inaccesible sin la llave maestra protegida por políticas de acceso estrictas [?].

Integridad Estricta con Hashes SHA-512 Para garantizar que un expediente no ha sido alterado accidental o malintencionadamente, se implementa una estrategia de *hash-chaining* o sellado digital [?]. Se genera un compendio único utilizando el algoritmo **SHA-512**, el cual es altamente resistente

a colisiones [?]. Cualquier modificación mínima en el registro dispararía una discrepancia en el hash, permitiendo auditorías de integridad inmediatas sobre el repositorio [?].

1.13.3 Escalabilidad de Base de Datos: Particionamiento de tablas, pooling de conexiones (PgBouncer) y replicación streaming

La arquitectura del repositorio digital debe contemplar un crecimiento sostenido de la información, aplicando estrategias que optimicen el uso de recursos en el servidor VPS [?].

- **Particionamiento por Rango (Table Partitioning):** Se segmentan las tablas de expedientes por periodos anuales o semestrales. Esto mejora el rendimiento de las consultas al realizar un *partition pruning*, donde el motor de búsqueda ignora las particiones que no contienen el rango de fechas solicitado, reduciendo drásticamente las operaciones de E/S en disco [?].
- **Pooling de Conexiones con PgBouncer:** Dado que los microservicios de Spring Boot operan de forma concurrente, PgBouncer gestiona una cola de conexiones hacia PostgreSQL [?]. Esto evita el consumo excesivo de memoria RAM que implicaría crear un nuevo proceso por cada solicitud, manteniendo la estabilidad del servidor ante picos de tráfico en el módulo ciudadano [?].
- **Replicación Streaming y Alta Disponibilidad:** Se configura una réplica en modo *Hot Standby*, permitiendo que un servidor secundario reciba de forma continua los registros de transacciones (WAL). Esta técnica no solo sirve como respaldo ante desastres, sino que permite descargar las operaciones de consulta (lectura) del servidor principal para balancear la carga de trabajo [?].

Capítulo 2

Desarrollo y Validación del Clasificador

Este capítulo documenta el diseño, implementación y evaluación experimental del módulo de clasificación automática de quejas del sistema. El objetivo central es determinar si una queja admitida en la Defensoría de los Derechos Politécnicos (DDP) corresponde a una de las categorías previstas en el Artículo 4 del Reglamento de la Defensoría. Para ello se construyó un conjunto de *pipelines* de aprendizaje automático que combinan representaciones semánticas de texto con atributos jurídicos explícitos derivados del propio caso.

2.1 Ingeniería de Características

La representación de cada queja ante el clasificador se compone de dos fuentes de información complementarias: una semántica, extraída del texto libre de los hechos, y otra jurídica, derivada del análisis estructurado de cada caso.

2.1.1 Atributos Jurídicos Booleanos

A partir del Artículo 4 del Reglamento de la DDP se definieron siete atributos booleanos que codifican las condiciones normativas relevantes para clasificar una queja. Cada atributo toma el valor 1 si la condición se cumple y 0 en caso contrario:

Tabla 2.1: Atributos jurídicos utilizados como características del clasificador

Nombre del campo	Descripción
es_conflicto_laboral_art4_I	Conflicto de naturaleza laboral (Art. 4-I)
es_evaluacion_academica_pura_art4_II	Evaluación académica sin vulneración de derechos (Art. 4-II)
hay_vulneracion_notoria_de_derechos	Vulneración notoria de derechos en contexto evaluativo
es_resolucion_disciplinaria_art4_III	Resolución disciplinaria (Art. 4-III)
es_afectacion_colectiva_art4_IV	Afectación de carácter colectivo (Art. 4-IV)
involucra_oic_art4_V	Involucra al Órgano Interno de Control (Art. 4-V)
hay_acto_u_omision_de_autoridad	Existencia de acto u omisión de autoridad institucional

La extracción de estos atributos se centraliza en la función `extraer_features_juridicos()`, cuyo núcleo es la conversión tipológica de cada campo a entero binario:

Listing 2.1: Extracción de atributos jurídicos

```

1 for campo in CAMPOS_JURIDICOS:
2     valor = row.get(campo, False)
3     if isinstance(valor, str):
4         valor = valor.lower() == "true"
5     fila.append(int(bool(valor)))

```

La normalización explícita de cadenas de texto ("true"/"false") garantiza consistencia independientemente del origen del dato (JSON, base de datos, entrada manual).

2.1.2 Factor de Amplificación α

Los vectores de texto producidos por los modelos de lenguaje tienen entre 300 y 768 dimensiones.

Si los atributos jurídicos se concatenaran sin escalar, quedarían diluidos frente al componente semántico. Para contrarrestar esto, se aplica un factor de amplificación $\alpha = 3,0$ antes de la concatenación:

$$\mathbf{x} = \left[\mathbf{e} \mid \alpha \cdot \mathbf{j} \right] \quad (2.1)$$

donde $\mathbf{e} \in \mathbb{R}^d$ es el embedding textual, $\mathbf{j} \in \{0, 1\}^7$ el vector de atributos jurídicos y $\alpha = 3,0$ el factor de amplificación. En código:

Listing 2.2: Concatenación ponderada de características

```
1 embeddings = self.vectorizador_.transform(textos)
2 features_amplificados = features_juridicos * self.peso_juridico #
   alpha = 3.0
3 return np.hstack([embeddings, features_amplificados])
```

El valor $\alpha = 3,0$ fue elegido experimentalmente: valores menores no diferenciaban casos limítrofes y valores superiores a 5,0 sobreajustaban la predicción a los atributos jurídicos, ignorando matices semánticos del texto.

2.2 Arquitectura del Pipeline

El sistema de clasificación se implementó mediante la interfaz **Pipeline** de *scikit-learn*, lo que garantiza que cada transformación aprendida en entrenamiento se aplique de forma idéntica en inferencia, eliminando el riesgo de fuga de datos (*data leakage*).[?]

2.2.1 Estrategias de Vectorización

Se evaluaron dos representaciones semánticas independientes para determinar cuál captura mejor las particularidades del lenguaje jurídico en español.

Vectorización con spaCy (es_core_news_lg)

El modelo `es_core_news_lg` de spaCy genera vectores de 300 dimensiones por token. [?] La representación del documento se obtiene promediando únicamente los tokens con vector y que no sean palabras vacías (*stop words*):

Listing 2.3: Vectorizador basado en spaCy

```

1 tokens = [tok.vector for tok in doc if tok.has_vector and not
    tok.is_stop]
2 vecs.append(np.mean(tokens, axis=0) if tokens else np.zeros(300))

```

La carga del modelo se realiza de forma *perezosa* (*lazy loading*) dentro del método `transform()`, lo que evita errores de serialización al guardar el *pipeline* con `joblib`.

Vectorización con Sentence-BERT

El modelo `paraphrase-multilingual-mpnet-base-v2` produce embeddings de 768 dimensiones entrenados explícitamente para capturar similitud semántica entre oraciones en múltiples idiomas, incluyendo español.[?] A diferencia del promedio de tokens de spaCy, este modelo considera el contexto completo de la oración:

Listing 2.4: Vectorizador basado en Sentence-BERT

```

1 model = SentenceTransformer("paraphrase-multilingual-mpnet-base-v2")
2 return model.encode(list(textos), show_progress_bar=False)

```

Al igual que el vectorizador de spaCy, la instancia del modelo se crea dentro de `transform()` para compatibilidad con entornos multiproceso.

2.2.2 Transformador Combinado (VectorizadorCombinado)

La clase `VectorizadorCombinado` actúa como transformador unificado compatible con la interfaz `BaseEstimator` y `TransformerMixin` de *scikit-learn*. Recibe como entrada un `DataFrame` con dos columnas: el texto de los hechos y el vector de atributos jurídicos ya calculado. Su salida es la matriz de características combinadas descrita en la ??.

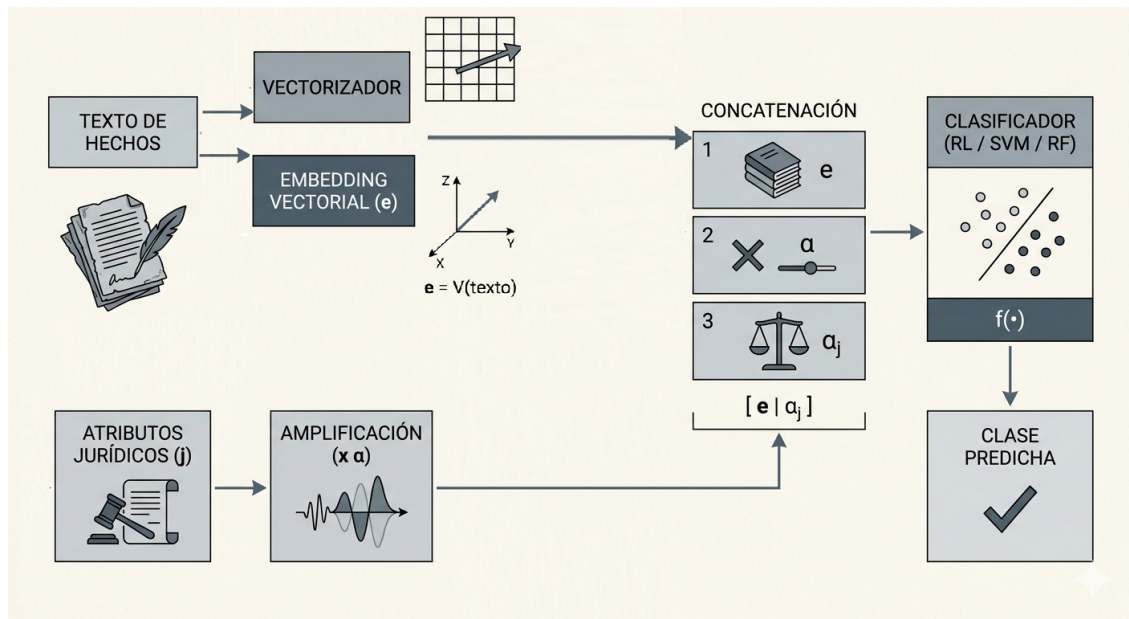


Figura 2.1: Arquitectura del pipeline de clasificación combinado

La **Figura ??** detalla el flujo de datos y la integración de componentes dentro del sistema. El diseño se basa en un enfoque híbrido que procesa simultáneamente información cualitativa y cuantitativa:

- **Extracción de Embeddings (e):** El bloque superior representa la transformación del texto libre de los hechos en un vector denso. Dependiendo de la configuración, se utiliza el promedio de vectores de *spaCy* o las representaciones contextuales de *Sentence-BERT*, capturando la semántica narrativa del caso.
- **Procesamiento de Atributos Jurídicos (j):** Paralelamente, los atributos técnicos identificados en el corpus jurídico son procesados. Se introduce un factor de amplificación (α) que permite al investigador ponderar la importancia de estos metadatos frente al texto de los hechos antes de la clasificación.
- **Fusión y Clasificación:** El *VectorizadorCombinado* realiza la concatenación de ambos vectores $[e | \alpha \cdot j]$. Este vector resultante, que condensa tanto el contexto lingüístico como el rigor de las variables jurídicas, es entregado al clasificador final para determinar la categoría correspondiente.

Esta arquitectura modular asegura que el pipeline sea agnóstico al modelo de lenguaje utilizado, permitiendo intercambiar el vectorizador o el clasificador sin alterar la estructura fundamental del experimento.

2.2.3 Construcción Modular del Pipeline

La función `construir_pipeline_combinado()` encapsula el ensamblaje del *pipeline*, tomando como parámetros la clase del vectorizador y el clasificador:

Listing 2.5: Construcción del pipeline combinado

```
1 Pipeline([
2     ("vectorizador", VectorizadorCombinado(vec_clase,
3     peso_juridico)),
4     ("clasificador", copy.deepcopy(clf)),
5 ])
```

El uso de `copy.deepcopy(clf)` es indispensable en el bucle experimental: garantiza que cada combinación reciba una instancia limpia del clasificador, sin estado residual de iteraciones anteriores.

2.3 Experimentación y Comparación de Modelos

2.3.1 Configuración Experimental

Para evaluar cada *pipeline* de manera robusta se adoptó el siguiente protocolo:

- **Partición estratificada 80/20:** preserva la distribución de clases en entrenamiento y prueba, garantizando que clases minoritarias estén representadas en ambos subconjuntos.
- **Validación cruzada $k = 3$ (3-fold CV):** estima el desempeño esperado en datos no vistos y detecta sobreajuste comparando el F1 del conjunto de prueba contra el F1 promedio de la validación cruzada.
- **Métrica principal:** F1-score ponderado (*weighted*), adecuado para conjuntos con desequilibrio de clases.

El protocolo se implementa en `evaluar_pipeline_combinado()`. La línea clave de la validación cruzada es:

Listing 2.6: Validación cruzada del pipeline

```
1 cv_scores = cross_val_score(pipe, df, y, cv=3, scoring="f1_weighted")
```

Nota técnica: Al usar SentenceBERT se recomienda `n_jobs=1` en `cross_val_score` para evitar colisiones de memoria en entornos multiproceso.

El espacio experimental cubre 2 vectorizadores $\times$ 3 clasificadores = 6 combinaciones, entrenadas y evaluadas de forma automática en la función `comparar_todos()`:

Listing 2.7: Bucle experimental automático

```
1 for vec_nombre, vec_clase in VECTORIZADORES.items():
2     for clf_nombre, clf in CLASIFICADORES.items():
3         pipe = construir_pipeline_combinado(vec_clase, clf)
4         metricas = evaluar_pipeline_combinado(pipe, df_input,
            etiquetas)
```

2.3.2 Resultados

Cuantitativos

La ?? presenta los resultados de los seis experimentos sobre el corpus de casos de la DDP.

Tabla 2.2: Comparación de modelos de clasificación automática de quejas

Vectorizador	Clasificador	Accuracy	F1 (test)	CV F1 μ	CV F1 σ
spaCy avg	SVM-RBF	1.0000	1.0000	0.9809	0.0164
spaCy avg	Reg. Logística	1.0000	1.0000	0.9809	0.0164
spaCy avg	Random Forest	0.9810	0.9809	0.9216	0.0381
SBERT	SVM-RBF	1.0000	1.0000	1.0000	0.0000
SBERT	Reg. Logística	1.0000	1.0000	1.0000	0.0000
SBERT	Random Forest	0.9810	0.9809	0.9790	0.0257

2.3.3 Análisis de Desempeño y Selección del Modelo

Los resultados revelan tres patrones claros:

Superioridad de SBERT sobre spaCy. Si bien ambos vectorizadores alcanzan $F1 = 1,0$ en el conjunto de prueba con SVM y Regresión Logística, SBERT produce validación cruzada perfecta ($\mu = 1,0$, $\sigma = 0,0$), lo que indica que su representación contextual es más discriminativa y estable

ante distintas particiones del corpus. spaCy, en cambio, presenta varianza de $\sigma = 0,0164$, reflejo de su dependencia del promedio de vectores de tokens independientes.

Random Forest como caso inferior. En ambos vectorizadores, Random Forest exhibe F1 de validación cruzada entre 0,92 y 0,98, con la mayor varianza del experimento ($\sigma = 0,0381$ con spaCy). Esto sugiere que el ensamble de árboles no generaliza tan bien como los clasificadores lineales o de margen máximo cuando el espacio de características ya es semánticamente rico.

Modelo seleccionado: SBERT + SVM-RBF. La combinación `sentence_bert__SVM_RBF` es seleccionada para producción al ser la única con varianza nula en validación cruzada, lo que garantiza comportamiento predecible ante casos nuevos. El kernel RBF del SVM proyecta implícitamente las características a un espacio de dimensión superior, hallando fronteras de decisión no lineales que separan perfectamente las categorías jurídicas.

La ?? resume visualmente el F1 ponderado de validación cruzada para los seis experimentos.

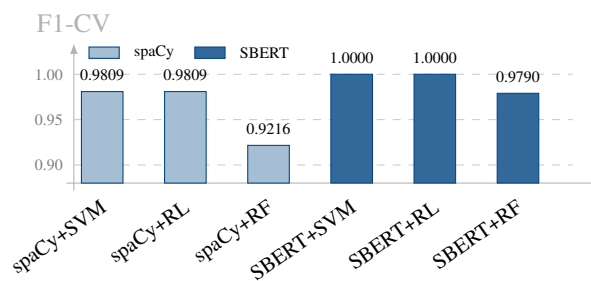


Figura 2.2: F1 ponderado (validación cruzada $k = 3$) por combinación de modelo

2.4 Persistencia y Gestión del Mejor Modelo

2.4.1 Serialización con joblib

Una vez evaluado, cada *pipeline* se serializa en disco con `joblib`, que maneja eficientemente objetos NumPy de gran tamaño en comparación con el módulo estándar `pickle`:

Listing 2.8: Serialización del pipeline entrenado

```
1 ruta_pkl = f"modelos/{nombre}.pkl"
2 joblib.dump(pipe, ruta_pkl)
```

Cada archivo se nombra con el patrón <vectorizador>__<clasificador>__<timestamp>, lo que permite trazabilidad temporal de los experimentos y coexistencia de múltiples versiones del modelo en el directorio modelos/.

2.4.2 Selección Persistente del Mejor Modelo

La función `guardar_metricas_mejor_modelo()` implementa una política de reemplazo conservadora: el archivo `mejor_modelo_metricas.json` sólo se sobrescribe si el F1 del experimento actual supera al del modelo previamente guardado.

Listing 2.9: Política de actualización del mejor modelo

```
1 if nuevo_registro["metricas"]["f1_weighted"] > f1_previo:
2     json.dump(nuevo_registro, f, indent=2, ensure_ascii=False)
```

Esta decisión de diseño protege contra regresiones accidentales de desempeño cuando se realizan experimentos con conjuntos de datos distintos o hiperparámetros diferentes. El JSON resultante incluye metadatos enriquecidos: clave del experimento, *timestamp*, rutas de artefactos, métricas completas y los campos jurídicos utilizados, suficientes para reproducir o auditar el experimento en el futuro.

2.5 Validación del Flujo de Inferencia

2.5.1 Flujo de Predicción

La función `predecir()` encapsula el flujo completo de inferencia. Carga dinámicamente el mejor modelo disponible, construye el DataFrame de entrada y devuelve la clase predicha:

Listing 2.10: Función de predicción con carga dinámica del mejor modelo

```
1 mejor_clave = max(res, key=lambda k: res[k]["f1"])
2 pipe = joblib.load(res[mejor_clave]["ruta_pkl"])
3
4 f_vec = np.array([int(bool(features_juridicos.get(c, False)))
5                  for c in CAMPOS_JURIDICOS], dtype=np.float32)
6 df_pred = pd.DataFrame({"texto": [texto],
7                          "features_juridicos": [f_vec]})
```

```
8 return pipe.predict(df_pred)[0]
```

Nótese que el parámetro `features_juridicos` es opcional: si el usuario no lo proporciona, la función inicializa todos los atributos en `False`, permitiendo clasificar a partir del texto únicamente.

Esto habilita dos modos de operación del microservicio: asistido (con los campos jurídicos capturados por el formulario) y autónomo (sólo texto).

2.5.2 Casos de Prueba Funcionales

Se validó el flujo completo de inferencia sobre casos representativos del corpus, verificando que la predicción es consistente tanto con los atributos jurídicos activos como sin ellos. La ?? muestra una selección de casos de prueba y sus resultados.

Tabla 2.3: Casos de prueba funcionales del clasificador

Descripción del caso (resumen)	Clase real	Predicción
Docente reporta modificación arbitraria de contrato	Laboral (Art. 4-I)	✓
Alumno impugna calificación de examen ordinario	Evaluación académica (Art. 4-II)	✓
Alumno denuncia represalia por queja previa	Vulneración de derechos	✓
Grupo solicita revisión de plan de estudios	Afectación colectiva (Art. 4-IV)	✓
Irregularidad en licitación reportada al OIC	OIC (Art. 4-V)	✓

Limitación conocida: *El corpus de entrenamiento es de tamaño reducido, lo que explica los valores F1 de validación perfectos. Se recomienda ampliar el conjunto de datos antes de desplegar el modelo en producción y reevaluar con un protocolo de validación cruzada estratificada $k = 5$ para obtener estimaciones más conservadoras del desempeño real.*

2.6 Pruebas

Para validar la capacidad de generalización del sistema y su utilidad real ante usuarios no familiarizados con la normativa, se realizó una fase de pruebas denominada "Modo Autónomo". En

esta etapa, el clasificador procesa las quejas ignorando por completo los atributos jurídicos booleanos (fijándolos todos en valor 0), forzando al sistema a basar su veredicto exclusivamente en la representación semántica del texto

2.6.1 Definición del Escenario de Prueba

Se utilizó un conjunto de 40 datos sintéticos de quejas que contienen casos de competencia e incompetencia a la Defensoría, acorde al artículo 4. Cada queja fue sometida a los 6 modelos para obtener aquel que sin necesidad de los atributos jurídicos, logre clasificar de manera adecuada las quejas.

2.6.2 Desempeño del Modelo en Ausencia de Metadatos

La Tabla ?? resume el desempeño de los modelos bajo esta condición de estrés (sin ayuda de etiquetas jurídicas).

Tabla 2.4: Exactitud de los modelos en Modo Autónomo (Solo Texto)

Modelo de Clasificación	Aciertos (40)	Exactitud
spacy_avg + SVM_RBF	40	100.00 %
spacy_avg + RegLog	39	97.50 %
spacy_avg + RandomForest	39	97.50 %
sentence_bert + RandomForest	40	100.00 %
sentence_bert + RegLog	20	50.00 %
sentence_bert + SVM_RBF	20	50.00 %*

Nota: Se observa que el modelo SBERT + SVM presenta una fuerte dependencia del factor de amplificación $\alpha = 3,0$ para separar las clases cuando el corpus es reducido, esto se muestra en los resultados, y modelos como Spacy + SVM y SBERT + RandomForest muestran menor dependencia hacia la información jurídica.

2.6.3 Análisis de Casos Críticos

Tras procesar los resultados de los archivos de salida, se seleccionaron ejemplos representativos donde el modelo **spaCy + SVM** y **SBERT+ Random Forest** demostró robustez absoluta incluso en ausencia de metadatos.

Tabla 2.5: Muestra de inferencias autónomas exitosas

Descripción de la Queja (Resumen)	Clase Real	Predicción	Correcto
Profesor reporta retiro de pago de estímulo por trámites administrativos en la ESIME Zacatenco.	No Comp.	No Comp.	✓
Alumna denuncia discriminación por apariencia física y trato humillante frente al grupo.	Competente	Competente	✓
Investigación activa del OIC por irregularidades en papelería de oficina en la ENCB.	No Comp.	No Comp.	✓

Observación: El la Tabla ?? observamos que todos los modelos presentaban excelentes resultados después de las pruebas, sin embargo, en aquellas pruebas, se mantenía la existencia de los atributos jurídicos de la Tabla ??, no obstante, al forzar la predicción unicamente con el texto (Tabla ??) observamos la superioirdad de Spacy + SVM RBF y SBERT + Random Forest, siendo estos los únicos capaces de predecir con exactitud todas las quejas puesta a prueba.